Institut für Kontinuumsmechanik

Leibniz Universität Hannover

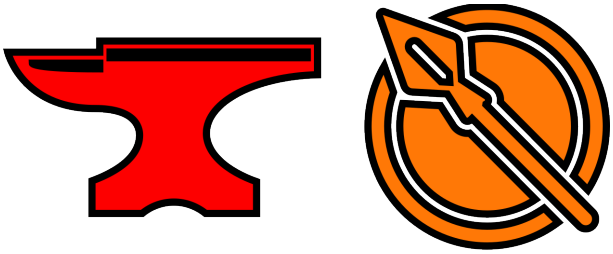
Leibniz Universität Hannover

Fakultät für Maschinenbau

Institut für Kontinuumsmechanik

Masterarbeit zum Thema

Programmierung einer Software zur Ableitung von Tensorgleichungen in Index-Notation



Tim Rother

Betreuer: Dr. Hendrik Geisler
1. Prüfer: Dr. Dustin R. Jantos
2. Prüfer: Dr. Matthias Wangenheim

31.03.2026

Angefertigt im Studiengang Maschinenbau am Institut für Kontinuumsmechanik der Leibniz Universität Hannover.

Eigenständigkeitserklärung

Hiermit erkläre ich, dass ich die am 31.03.2026 eingereichte studentische Arbeit zum Thema „Programmierung einer Software zur Ableitung von Tensorgleichungen in Index-Notation" unter Betreuung von Dr. Hendrik Geisler selbstständig erarbeitet, verfasst und Zitate kenntlich gemacht habe. Weiterhin erkläre ich, dass ich keine anderen als die angegebenen Hilfsmittel verwendet habe.

Zur Anfertigung der vorliegenden Arbeit wurden Werkzeuge basierend auf generativer künstlicher Intelligenz zur Informationsgewinnung und Literaturverarbeitung eingesetzt. Die sich daraus ergebenen Informationen und Quellen habe ich nach bestem Wissen und Gewissen geprüft. Werkzeuge basierend auf generativer künstlicher Intelligenz wurden zur Korrektur und Überarbeitung der sprachlichen Formulierung ohne weitere inhaltliche Anpassungen genutzt. Die durch die künstliche Intelligenz sprachlich überarbeiteten Texte habe ich auf inhaltliche Abweichungen von meiner persönlichen ursprünglichen Formulierung geprüft und korrigiert.

Hannover, 31.03.2026 _____

Tim Rother

Inhaltsverzeichnis

Eigenständigkeitserklärung i

1 Einleitung 1

2 Zielsetzung 1
2.1 Anforderungsprofil 1
2.2 Konzept 2
2.2.1 Eingabeformat 2
2.2.2 Steuereinheit 3
2.2.3 Tensoralgebra-Backend 3
2.2.4 Ergebnisexport 3

3 Implementierung 4
3.1 Projektstruktur 4
3.1.1 Teilprojekt Alberich 4
3.1.2 Teilprojekt Baldur 5
3.2 Alberich 6
3.2.1 Laufzeitentitäten 6
3.2.1.1 Objekte 6
3.2.1.2 Variablen 8
3.2.1.3 Scopes 9
3.2.1.4 Evaluierungsergebnisse 9
3.2.2 Sprachregister 10
3.2.2.1 Typenregister 10
3.2.2.2 Keywordregister 10
3.2.2.3 Funktionsregister 11
3.2.2.4 Backendregister 12
3.2.2.5 Registerinstanzen 12
3.2.3 Interpreter 12
3.2.3.1 Präprozessor 13
3.2.3.2 Lexer 13
3.2.3.3 Parser 14
3.2.3.4 Evaluierer 15
3.2.3.4.1 Rückgabe 16
3.2.3.4.2 Evaluierungsroutine 16
3.2.3.4.3 Überladungskonfiguration 17
3.2.3.4.4 Steuerstatements 18
3.2.3.4.5 Kontexte 19
3.2.3.4.6 Fehlerbehandlung 19
3.3 Baldur 20
3.3.1 Überladungskonfiguration 20
3.3.2 Basistypen 20
3.3.3 Konzipierter Ablauf 21
3.3.4 Ausdruckskonventionen 21
3.3.5 Algebraische Operationen 21

3.3.6 Ausdrucksassemblierung . . . 23
3.3.7 Templatesystem . . . 23
3.3.8 Ableitungen . . . 25
3.3.9 Vereinfachungen . . . 27
3.3.10 Ergebnisexport . . . 27
3.3.11 Language Support . . . 28

4 Anwendung . . . 29
4.1 Alberich . . . 29
4.1.1 Distributionsherstellung . . . 29
4.1.1.1 Backendfreischaltungen . . . 29
4.1.1.2 Backendregistrierung . . . 31
4.1.1.3 Skriptausführung . . . 31
4.1.1.4 Interpreterprogramm . . . 32
4.1.1.5 Einbindung in bestehende digitale Ökosysteme . . . 32
4.1.2 Sprachreferenz . . . 33
4.1.2.1 Skriptformatierung . . . 33
4.1.2.2 Standardobjekte . . . 34
4.1.2.3 Objekt Konstruktion . . . 34
4.1.2.4 Simultanverknüpfungen . . . 35
4.1.2.5 Berücksichtigung der Operatorenhierarchie . . . 36
4.1.2.6 If-Syntax . . . 36
4.1.2.7 Speicherverwaltungssemantik . . . 37
4.1.2.8 Funktionsdeklarierung . . . 38
4.1.2.9 Ausgaben . . . 39
4.1.2.10 Guards . . . 39
4.1.2.11 Schleifen . . . 40
4.1.2.12 Objektorientierte Programmierung . . . 40
4.1.2.13 Assertions . . . 41
4.1.2.14 Einbinden von Inhalten . . . 42
4.1.2.15 Metavariablen . . . 42
4.1.2.16 Diagnostik . . . 43
4.1.2.17 Fehlerbehandlung . . . 44
4.2 Baldur . . . 45
4.2.1 Installation . . . 45
4.2.2 Konstruktion algebraischer Tensorausdrücke . . . 45
4.2.3 Verknüpfung algebraischer Ausdrücke . . . 46
4.2.4 Konventionssteuerung . . . 47
4.2.5 Ersetzungen . . . 47
4.2.6 Templatesystem . . . 48
4.2.7 Ableitungen . . . 48
4.2.8 Vereinfachungen . . . 50
4.2.9 Indexnotierte Tensorausdrücke . . . 51
4.2.10 Julia-Skript-Generierung . . . 52
4.2.11 Potenzielle Limitierungen . . . 53
4.2.12 Fehlerbehandlung . . . 53

5 Leistungsbeurteilung . . . 54

5.1 Auswertung Beispielimplementierungen 54
5.2 Vergleich mit vorhandenen Implementierungen 54
5.3 Zusammenfassung 55
5.4 Ausblick 55

Abbildungsverzeichnis

1 Arbeitsweise des Werkzeugs 2
2 Logo des Projekts Alberich 4
3 Logo des Projekts Baldur 4
4 Verarbeitungskette des Interpreters 13

Tabellenverzeichnis

1 Verschiedene template-ierte Knotentypen des Templatesystems 23
2 Beispiele für Templateüberladungen nach verschiedenen Faktoren 25
3 Standard Makros für Funktionsregistrierung 30
4 Freigeschaltete Diagnostikfunktionen 43
5 Konstruktion von externen, algebraischen Tensorausdrücken 45
6 Implementierte algebraische Operationen 46
7 Freigeschaltete Funktionen zur Konventionssteuerung 47
8 Freigeschaltete Funktionen für Ausdruckssubstitution 47
9 Funktionen zur Verwaltung der Ableitungskonfiguration 49
10 Funktionen zur Steuerung der Vereinfachungskonfiguration 50
11 Evaluierungsstatistiken nach TSM-Stufe 54
12 Vergleich zwischen AceGen- und Baldur-Implementierungen 55

Algorithmenverzeichnis

1 Aufbau des polymorphen Typen-Interfaces 6
2 Aufbau des indizierten polymorphen Typen-Interfaces 7
3 Aufbau des nativen polymorphen Typen-Interfaces 7
4 Aufbau der Variablen Struktur 8
5 Aufbau des Scopes 9
6 Aufbau des Evaluierungsergebnisses 9
7 Aufbau des Typenregisters 10
8 Von Funktionsregister erwartetes Funktionsformat 11
9 Aufbau des Funktionsregisters 11
10 Aufbau des Lexer-Tokens 14
11 Aufbau des Parser-AST-Knotens 14
12 Einfluss der Operatorenhierarchie auf die Evaluierung 15
13 Aufbau des ProcessingResults 16
14 Funktionsweise der rekursiven Evaluierungsroutine 16
15 Exemplarische Verknüpfung einer Operandenkette 18
16 Angabe der Überladungskonfiguration 18
17 Kontextevaluierung und -mitteilung in der Evaluierungsroutine 19
18 Anschließen der Fehlerkontextfunktion 20

19 Aufbau der symbolischen Darstellung algebraischer Tensorausdrücke 21
20 Funktionsweise algebraischer Operationsimplementierungen 22
21 Funktionsweise der tensoralgebraischen Ausdrucksableitung 26
22 Implementieren einer Typenklasse 29
23 Registrieren einer Typenklasse und Funktion 31
24 Registrieren eines Backends 31
25 Anlegen eines Distributionsscopes und Weitergabe an den Interpreter 32
26 Übersicht über die erwartete Skriptformatierung 34
27 Explizite und implizite Konstruktion von Instanz- und Referenzvariablen 35
28 Übersicht über die Nutzungsmöglichkeiten von Simultanverknüpfungen 35
29 Übersicht über die If-Syntax 37
30 Übersicht über explizite und implizite Speicherverwaltungssemantik 38
31 Korrekte Handhabung durch LValue Verschiebung invaliderter Ressourcen 38
32 Funktionsdeklaration und Pseudo-Lambda Syntax 39
33 Übersicht über die Ausgabefunktionen 39
34 Verwendung von Main- und Include-Guards 40
35 Übersicht über verwendete Schleifenformate 40
36 Beispielhafte Verwendung der For-Schleife 40
37 Beispiel für Alberichs Objektorientierte Programmierung 41
38 Übersicht über Assertionsformate 42
39 Einbinden externer Ressourcen in Distributionsskripte 42
40 Abspeichern von Metavariablenwerten in einem Pseudonamespace 43
41 Beispielhafte Interpreterfehlermeldung 44
42 Übersicht über Templatediagnostikfunktionen 48
43 Ableitungs-Beispiel 50
44 Vereinfachungsbeispiel 51
45 Konvertierung von algebraischen zu indexnotierten Ausdrücken 52
46 Beispiel für Julia-Skript-Generierung 52

1 Einleitung

Die algebraische Tensorrechnung ist ein zentrales Element vieler wissenschaftlicher Disziplinen, wie der Kontinuumsmechanik, der Wärmeübertragung, der Thermodynamik, der Astro- und der Quantenphysik. Das Ableiten von komplexeren Tensorausdrücken erfordert dabei häufig einen computergestützten Arbeitsprozess. Herkömmliche Tensoralgebra-Softwarelösungen sind dafür teilweise nur begrenzt geeignet. Die Beschränkungen sind unter anderem ein sehr begrenztes Potenzial zur Verarbeitung von Ausdrücken, die eine gewisse Komplexität überschreiten, lange Rechenzeiten, hohe Hardwareanforderungen oder eine sehr ausgeprägte Spezialisierung auf einzelne Anwendungsgebiete. Deshalb soll im Rahmen dieser Arbeit ein Werkzeug entwickelt werden, das das Aufstellen und Ableiten symbolischer Tensorausdrücke sowie deren Umwandlung in Indexnotation anwendungsunabhängig ermöglicht und dabei angemessene Rechenzeiten auf handelsüblicher Hardware bietet. Zudem soll das Werkzeug minimale Abhängigkeiten haben und eigenständig nutzbar, leicht installierbar und intuitiv bedienbar sein. Des Weiteren soll das Programm über eine nicht-kommerzielle Veröffentlichung allgemein zugänglich gemacht werden. Neben Darstellungen und Erläuterungen der ausgearbeiteten Konzepte, verwendeter Algorithmen und der Umsetzung und Implementierung dieser Inhalte, wird diese Arbeit konkrete Anleitungen zur effizienten Nutzung des implementierten Werkzeugs, inklusive mehrerer Beispiele und Demonstrationen sowie der Ergebnisse von Leistungsmessungen enthalten.

2 Zielsetzung

In dem folgenden Kapitel werden die Anforderungen an das Werkzeug, das im Rahmen dieser Arbeit entwickelt werden soll, formuliert und ein Konzept zum Aufbau und der Funktionsweise des Programms erstellt.

2.1 Anforderungsprofil

Das folgende Unterkapitel stellt das Anforderungsprofil dar. Der Titel der Arbeit formuliert bereits die primäre Anwendung des Werkzeugs, das Aufstellen und Ableiten von Tensorausdrücken und die Konvertierung der Ausdrücke in Indexnotation. Dabei soll der Nutzer Vorgaben über ein flexibles Eingabesystem tätigen können. Die evaluierten, indexnotierten Ergebnisse müssen exportierbar und aus anderen Werkzeugen und Umgebungen heraus nutzbar sein. Zudem soll das Programm auch mit komplexeren Ausdrücken auf handelsüblicher Hardware innerhalb akzeptabler Laufzeiten rechnen können. Eine unkomplizierte Einrichtung und Einarbeitung in das Werkzeug wird angestrebt.

Primäre Anforderungen

- Aufstellen und Manipulieren von algebraischen Tensorausdrücken
- Umwandlung algebraischer Ausdrücke in Indexnotation
- Export indexnotierter Ausdrücke
- Programmsteuerung über flexibles Eingabesystem
- Akzeptable Rechenzeiten auf gängiger Hardware
- Unkomplizierte Installation und Einarbeitung

2.2 Konzept

Um den in Abbildung 2.1 formulierten Anforderungen gerecht zu werden, wird ein einfaches Konzept für den Aufbau des Werkzeugs ausgearbeitet. Dieses sieht ein eigenständiges Programm vor, das aus einem Tensoralgebra-Backend und einer Steuereinheit besteht. Diese Steuereinheit nimmt die Vorgaben des Nutzers in Form eines verständlichen, flexibel einsetzbaren Eingabeformats entgegen, wertet es aus, erstellt und verwaltet die Laufzeitobjekte, auf denen das Tensoralgebra-Backend arbeiten soll und setzt Befehle an das Tensoralgebra-Backend ab. Das Backend, das die Laufzeitobjekte implementiert, manipuliert diese den Anweisungen der Steuereinheit entsprechend. Abbildung 1 zeigt diesen Aufbau exemplarisch.

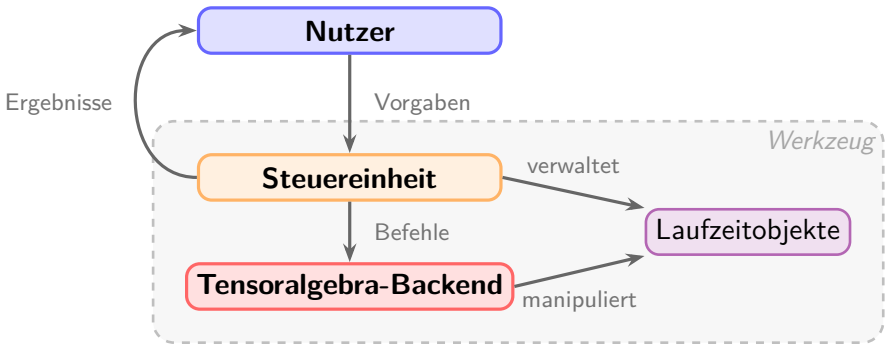


Abbildung 1: Arbeitsweise des Werkzeugs

2.2.1 Eingabeformat

Das Eingabeformat ist hier das Protokoll, über das der Nutzer die Steuereinheit des Werkzeugs direkt anspricht. Anhand entsprechend formatierter Anweisungen wird der Steuereinheit mitgeteilt, wie sie die Laufzeitobjekte des Backends verwalten soll und welche Befehle sie an das Backend absetzt. Dabei soll das Eingabeformat intuitiv gehalten werden und eine schnelle Einarbeitung ermöglichen. Möglichkeiten ein solches Format umzusetzen sind

- Ein grafisches Eingabeformat
- Ein dateibasiertes Eingabeformat

Grafische Eingabeformate lassen sich sowohl über Benutzeroberflächen realisieren, als auch mittels visueller Systeme, wie zum Beispiel Editierumgebungen für knotenbasierte Strukturbäume. Ansätze dateibasierte Formate umzusetzen sind dedizierte, domänenspezifische Skriptsprachen (DSL), wie LaTeX (Lamport TeX), HTML (HyperText Markup Language) oder CMake (Crossplatform Make), oder Konfigurationsformate, wie INI (Initialization) oder JSON (JavaScript Object Notation). Benutzeroberflächen und Konfigurationsdateien bieten häufig ein nur sehr begrenztes Potenzial für die Übermittlung komplexerer Routinen und Anweisungspaletten. Für diese Anwendungsfälle sind dedizierte Skriptsprachen und visuelle Systeme besser geeignet. Sowohl visuelle Formate als auch DSLs erfordern einen gewissen Einarbeitungsaufwand. Dieser kann im Rahmen einer Skriptsprache jedoch reduziert werden, indem die Syntax an die gebräuchliche, tensoralgebraische Schreibweise angelehnt wird. Zudem erfordert eine DSL im Gegensatz zu einem visuellen Eingabeformat keine dedizierte grafische Anwendung, sondern kann in andere digitale Ökosysteme und Umgebungen wie Codeeditoren oder IDEs (Integrated Development Environment) implementiert werden. Um eine möglichst flexible Steuerung des Prozesses zu

gewährleisten und eine intuitive, schnell erlernbare Eingabe zu ermöglichen, die sich an der gängigen Tensoralgebra-Syntax orientiert, wird im Rahmen dieser Arbeit eine domänenspezifische Skriptsprache als Eingabeformat implementiert.

2.2.2 Steuereinheit

Die Steuereinheit ist im Rahmen dieser Architektur die zentrale Schnittstelle zwischen dem Nutzer und den performanten, nativen Programmressourcen, wie dem Tensoralgebra-Backend. Der Nutzer formuliert eine Anweisungssequenz in Form des verwendeten Eingabeformats. Die Steuereinheit bekommt diese Anweisungssequenz übergeben und interpretiert sie. Dabei verwaltet die Einheit die Behandlung der Laufzeitobjekte den vom Nutzer formulierten und übergebenen Anweisungen entsprechend. Im Rahmen dieser Arbeit wird ein spezialisierter Interpreter als Steuereinheit implementiert, der das tensoralgebraische DSL-Eingabeformat liest und interpretiert.

2.2.3 Tensoralgebra-Backend

Das Tensoralgebra-Backend implementiert die algebraischen und indexnotierten Tensorausdrücke und stellt Funktionen stellt, mit denen diese Objekte manipuliert werden können. Um das Backend zur Laufzeit steuern zu können, greift die Steuereinheit beim Interpretieren der vom Nutzer formulierten Anweisungspalette auf die Funktionalitäten des Backends zurück, erstellt und löscht Instanzen dieser Objekte und ruft die vom Backend zur Verfügung gestellten Funktionen auf, um die Instanzen zu manipulieren. Dabei werden sowohl die algebraischen, als auch die indexnotierten Ausdrücke symbolisch repräsentiert.

2.2.4 Ergebnisexport

Der Ergebnisexport dient primär der Nutzung der ermittelten Ergebnisse aus anderen Programmen und digitalen Ökosystemen heraus. Dabei werden die indexnotierten Ausdrücke in programmexterne Dateiformate übersetzt. Da mit dem Werkzeug erzeugte Ergebnisse zuerst im Rahmen von in Julia implementierten Finite-Elemente-Analysen zum Einsatz kommen sollen, wird zunächst der Export von indexnotierten Ausdrücken über eine Julia-Skript-Generierung implementiert.

3 Implementierung

In diesem Kapitel wird die Implementierung des in Kapitel 2.2 formulierten Konzepts beschrieben.

3.1 Projektstruktur

Um die Übersichtlichkeit und Wiederverwendbarkeit einzelner Programmbestandteile zu gewährleisten, wird das Projekt in zwei Teilprojekte unterteilt, die Projekte Alberich und Baldur. Der Name Alberich ist dabei ein Akronym für den Ausdruck **A**ssembling **L**ibrary for **B**uilding **E**mbedded **R**untime and Independent **I**nterpreters **C**ontrolling native **H**igh Performance Ressources. Dieses Teilprojekt ist eine Bibliothek zum Herstellen von eingebetteten und eigenständigen Interpretern, die native, performante Programmressourcen dynamisch verwalten. Damit lassen sich leichtgewichtige, dedizierte Interpretersprachen schreiben, die dem Nutzer die dynamische Steuerung von C- und C++-Backends und Echtzeitmodifizierungen performanter Programme ermöglichen. Alberich implementiert dabei eine gleichnamige Grundsprache. Das Herstellen der konkreten Distribution dieser Grundsprache erfolgt über das Implementieren von Treiberfunktionen, die native Backendtypen und Funktionen für die Nutzung innerhalb der Sprachdistribution freischalten. Der Name Baldur ist ein Akronym für **B**asic **A**lgebraic **L**anguage for **D**eriving **U**nified Tensor **R**epresentations und das eigentliche Zielwerkzeug dieser Arbeit. Dieses Projekt nutzt die ebenfalls im Rahmen dieser Arbeit implementierte Bibliothek Alberich, um eine dedizierte Interpretersprache herzustellen. Damit ist Baldur eine Distribution der Grundsprache Alberich. Baldur implementiert ein eigenes symbolisches Tensoralgebra-Backend und schaltet es über die gleichnamige Sprachdistribution dem Nutzer frei. Die gewählten Logos der Teilprojekte werden in Abbildung 2 und Abbildung 3 gezeigt.

Abbildung 2: Logo des Projekts Alberich

Abbildung 3: Logo des Projekts Baldur

3.1.1 Teilprojekt Alberich

Das Teilprojekt Alberich implementiert eine Grundsprache und eine Pipeline zum Freischalten von Backendtypen und -funktionen. Die Entscheidung mit Alberich eine Skriptsprache von Grund auf zu implementieren und keine bestehende Lösung zu nutzen hat mehrere Gründe. Zum einen verfügen die meisten Interpretersprachen, die sich für eine Laufzeitmodifizierung eignen, wie beispielsweise Python, Lua und ChaiScript, über eine begrenzte Operatorenauswahl. Diese Operatoren sind in erster Linie für skalare, boolesche oder bitweise Operationen vorgesehen. In diesem Projekt soll sich die Syntax der DSL jedoch stark an der gängigen Schreibweise der Tensoralgebra orientieren. Die Verwendung eines bestehenden Frameworks zum Herstellen des Interpreters ist zwar möglich, geschieht allerdings auf Kosten einer intuitiven und gut lesbaren, tensoralgebraischen Syntax. Zum anderen bieten die üblichen Lösungen häufig keine Möglichkeit einer expliziten Speicherverwaltung innerhalb der hergestellten Interpretersprache. Für das Arbeiten mit den symbolischen Ausdrücken empfiehlt sich diese explizite Speicherverwaltungssyntax jedoch. Die algebraischen Ausdrücke werden als abstrakte Syntax Bäume (AST)

repräsentiert. Dabei handelt es sich um rekursive Speicherstrukturen, die für Ausdrücke der erwarteten Komplexität sehr groß werden und sich tief verzweigen können. Eine aufgrund einer impliziten Syntax unbeabsichtigt durchgeführte, suboptimale Speicherverwaltung kann dabei massive Leistungseinbußen verursachen. Zudem bringen übliche Lösungen häufig umfangreiche Abhängigkeiten mit, die das Programm und die ausführbare Datei aufblähen. Aus diesen Gründen implementiert Alberich einen eigenen Interpreter mit leichtgewichtigem Präprozessor, Lexer, Parser und Evaluierer. Dabei kann eine beliebige Operatorenauswahl genutzt werden, die durch die Sprachdistribution konfiguriert wird. Zudem bietet Alberich die Möglichkeit eine sehr explizite Speicherverwaltungssyntax zu schreiben, die eine effiziente Handhabung großer symbolischer Tensorausdrücke unterstützt.

3.1.2 Teilprojekt Baldur

Das Teilprojekt Baldur implementiert ein Tensoralgebra-Backend und eine Alberich-Sprachdistribution, die dieses Backend freischaltet. Die Tensorausdrücke werden dabei durch AST-Strukturen repräsentiert. Das Tensoralgebra-Backend implementiert tensoralgebraische Operationen wie Skalarprodukte, das dyadische Produkt und die Ableitung. Um dem Nutzer die Möglichkeit zu geben das Backend und seine Funktionsweise fallspezifisch zu konfigurieren und zu optimieren, implementiert Baldur mehrere Register für spezifische Ableitungs- und Vereinfachungsregeln. Diese Regeln müssen dabei nicht zwangsläufig instanzspezifisch sein, sondern können über ein mitgeliefertes Template-System verallgemeinert werden. Dieses Template-System kommt dabei mit verschieden Template-Ausführungen, die verschiedene Arten von Ausdrücken repräsentieren und einer Überladungslogik, damit Vorgaben für allgemeinere Templates durch spezifischere Templates oder Instanzen noch spezifiziert werden können. Damit können sowohl verschieden allgemeine als auch instanzspezifische Ableitungs- und Vereinfachungsregeln implementiert werden.

3.2 Alberich

Dieses Kapitel beschreibt die Implementierung der Bibliothek Alberich. Dabei wird sowohl auf die Freischaltungs-Pipeline, als auch die Umsetzung des Alberich-Interpreters eingegangen.

3.2.1 Laufzeitentitäten

Die Laufzeitentitäten sind dabei die Objekte, mit denen der Interpreter arbeitet. Sie dienen der Darstellung und Verwaltung konkreter Instanzen freigeschalteter Backendobjekte.

3.2.1.1 Objekte

Das Typensystem von Alberich basiert auf der polymorph genutzten Klasse `IObject`. Dabei definiert `IObject` mehrere gelöschte, virtuelle Funktionen, die der Definition der objektspezifischen Interpreter-Interaktion dienen. Da diese Funktionen als gelöscht deklariert werden, können keine direkten Instanzen des `IObject` erzeugt werden. Aus diesem Grund wird diese Klasse als abstrakte Klasse oder Interface bezeichnet. Die vorgesehene Nutzung des `IObjekt` ist das Beerben einer Kindklasse, die diese virtuellen Interaktionsfunktionen überschreibt. Instanzen einer solchen Kindklasse, die von einer abstrakten Elternklasse erbt, werden im Folgenden als polymorphe Instanzen bezeichnet. Die polymorphen Instanzen des `IObject` entsprechen den konkreten Werten oder Objekten mit denen der Interpreter arbeitet. Deshalb werden die Kindklassen, die das `IObject` beerbt im Folgenden als Typen bezeichnet. Die konkrete polymorphe Nutzung des `IObject` erfolgt über Zeiger. Dabei werden polymorphe Instanzen vereinheitlicht als `IObject`-Zeiger gespeichert. Durch die Vereinheitlichung verliert der Interpreter die makroskopische Information über den Typen der polymorphen Instanz. Die individuelle, typenspezifische Behandlung erfolgt aufgrund der typenspezifischen Überschreibungen, der virtuellen Interaktionsfunktionen dennoch. Der Algorithmus 1 zeigt eine exemplarische Klassendeklaration für die abstrakte Elternklasse `IObject`.

Algorithmus 1: Aufbau des polymorphen Typen-Interfaces

```
1  class IObject{
2
3    // Virtueller Dekonstruktor
4    virtual ~IObject() = default;
5
6    // Virtuelle, gelöschte Funktionen die Behandlung des Objekts definieren
7    virtual void print() const = 0;
8    virtual std::unique_ptr<IObject> clone() = 0;
9
10   // Typenidentifizierung
11   virtual TypeIndex getTypeIndex() const = 0;
12   virtual const std::string& getTypeKeyword() const = 0;
13 };
```

Um die einzelnen Typen trotz der polymorphen Vereinheitlichung zur Laufzeit unterscheiden zu können, werden sie unter verschiedenen Indizes registriert. Um diese Typenregistrierung zu automatisieren und den Boilerplate-Code zu reduzieren, der für jede Kindklasse geschrieben werden muss, wird die abstrakte Klasse `IIndexedObject` implementiert. Dieses Interface erbt dabei vom `IObject`. Damit sind polymorphe Instanzen des `IIndexedObject` ebenfalls polymorphe Instanzen des `IObject` und können als Teil des Typensystems vereinheitlicht werden. Das `IIndexedObject` implementiert dabei die für die Registrierung benötigte Grundstruktur. Zudem wird mit dem Überschreiben mehrerer virtueller Funktionen des `IObject` die typenspezifische Behandlung der

polymorphen Instanzen spezifiziert. Algorithmus 2 zeigt eine exemplarische Klassendeklaration für die abstrakte Elternklasse `IIndexedObject`, die die Typenregistrierung automatisiert. Die Templategröße `Tag` ist dabei eine Signatur, die die konkrete Unterscheidung von template-ierten Erben mit gleichen Templateargumenten ermöglicht. Für den `Tag` sollte also ein kindklassenspezifisches Argument eingesetzt werden. Dabei kann die Kindklasse selbst direkt als `Tag` verwendet werden.

Algorithmus 2: Aufbau des indizierten polymorphen Typen-Interfaces

```
1 template<typename Tag>
2 class IIndexedObject : public IObject{
3
4   static TypeIndex typeIndex;
5
6   TypeIndex getTypeIndex() const override { return typeIndex; }
7   const std::string& getTypeKeyword() const override { return getKeywordByTypeIndex(typeIndex); };
8
9   static bool init(const std::string& keyword, const std::function<IObject*()>& initConstructor){
10
11     typeIndex = registerType(keyword, initConstructor);
12     RETURNING_ASSERT(typeIndex != INVALID_TYPE_INDEX, "...", false)
13
14     return true;
15   }
16 };
```

Die Interfaces `IObject` und `IIndexedObject` ermöglichen bereits die Definition und Registrierung von Laufzeitobjekttypen. Dabei kann die Behandlung dieser Objekte für jeden Typen individuell anhand von Überschreibungen der virtuellen Funktionen des `IObject` deklariert werden. Die Freischaltung einer konkreten Backendobjektinstanz pro Objekt kann dabei zusätzlich automatisiert werden. Dazu wird das Interface `INativeObject` implementiert. Dieses verpackt genau einen Backendtypen und schaltet ihn für die Nutzung in der Distribution frei. Zudem ermöglicht es die Uniformisierung von Instanzen dieses Objekttypen. Von Uniformisierung wird hier gesprochen, wenn der dedizierte Interpreter die Objektinstanz nicht selbst verwaltet, sondern eine bestehende Instanz freischaltet. So können bestehende Programmressourcen dynamisch und ohne jeglichen Übersetzungsaufwand manipuliert werden. Die Uniformisierung erlaubt damit die unkomplizierte Echtzeitmodifizierung performanter Software über einen eingebetteten, dedizierten Interpreter. Dabei bietet Alberich den Vorteil einer sehr simplen, leicht verständlichen und schnell schreibbaren Syntax, die zur Laufzeit interpretiert werden kann und dabei flexibel die Ausführung nativer, performanter Funktionalitäten des freigeschalteten Backends ermöglicht. In Algorithmus 3 ist die exemplarische Deklaration des `INativeObject` Interfaces dargestellt. Für die Verwendung dieser abstrakten Klasse ist es erforderlich, dass der `Tag` die Kindklasse ist, die den Backendtypen freischaltet. Das liegt daran, dass der `Tag` in einigen Funktionen, wie dem Kopierkonstruktor des `INativeObject` zum Einsatz kommt. Um zu vermeiden, dass dieser Vorgang in jeder Kindklasse noch einmal gepackt werden muss wird die kopierte Instanz direkt als Instanz der Kindklasse konstruiert.

Algorithmus 3: Aufbau des nativen polymorphen Typen-Interfaces

```
1 template<typename Tag, typename T>
2 class INativeObject : public IIndexedObject<Tag>{
3
4   bool IsManagingMemory = true;
5   T* member;
```

```
6
7   // Konstruktoren für verwaltete Objekte
8   INativeObject(){ member = new T(); }
9   INativeObject(const T& memberIn){ member = new T(memberIn); }
10
11  // Konstruktor für uniformisierte Objekte
12  INativeObject(T* memberPtrIn) : member(memberPtrIn){ IsManagingMemory = false; }
13
14  ~INativeObject(){ if(IsManagingMemory){ delete member; } }
15  bool isUniform() override { return !IsManagingMemory; }
16 };
```

3.2.1.2 Variablen

Die in Kapitel 3.2.1.1 beschriebenen abstrakten Klassen ermöglichen bereits die Freischaltung, Registrierung und Uniformisierung von Backendobjekten. Die Verwaltung dieser freigeschalteten, vereinheitlichten Objekte realisiert der Interpreter dabei über Variablen. Variablen sind dabei referenzierbare Instanzen, die Objekte besitzen, oder Referenzen, die die Objekte von Instanzvariablen referenzieren. Um sicherzustellen, dass eine Objektinstanz nur einmal von einer Variablen besessen werden kann, wird die polymorphe Nutzung der Objekte innerhalb der Variablen über den Smart-Pointer `std::unique_ptr` realisiert. Ein `unique_ptr` ist dabei ein Zeiger auf ein Objekt, der für diese Instanz gleichzeitig nur einmal existieren kann. Da der `unique_ptr` nur ein Wrapper des Standardzeigers ist, ist er ebenfalls für die polymorphe Nutzung von abstrakten Klassen geeignet. Die Variable enthält dabei einen `unique_ptr` für das besessene Objekt und einen `unique_ptr*` für das referenzierte Objekt. Ob die Variable eine Instanz, Referenz oder ungültig ist, lässt sich dabei anhand der Gültigkeit der entsprechenden Zeiger erkennen. Der Algorithmus 4 demonstriert diesen Aufbau der Variable.

Algorithmus 4: Aufbau der Variablen Struktur

```
1 struct Variable{
2
3     std::unique_ptr<IObject> ownedObject = nullptr;         // Besessenes Objekt
4     std::unique_ptr<IObject>* referencedObject = nullptr;   // Referenziertes Objekt
5
6     bool isReference() const { return ownedObject == nullptr; }
7     void reference(const Variable& other){ referencedObject = &other.ownedObject; }
8 };
```

Vor dem Löschen einer Instanzvariablen müssen alle zugehörigen Referenzvariablen dereferenziert werden, ansonsten kommt es zu einer Speicherzugriffsverletzung. Das Programm versucht auf einen Speicher zuzugreifen, den es bereits freigegeben und auf den es keine Zugriffsberechtigung mehr hat. Bei einer solchen Zugriffsverletzung wird das Programm automatisch terminiert. Aus diesen Gründen bietet Alberich umfangreiche Dereferenzierungsroutinen, die beim Löschen einer Instanzvariablen eigenständig nach Referenzen der Instanz suchen, diese dereferenzieren und den Nutzer über eine Fehlermeldung auf eine nicht rechtzeitig derefenzierte Referenzvariable hinweisen. Da der Overhead dieser Routinen linear mit der Gesamtanzahl der Variablen skaliert, kann es in manchen Fällen vorteilhaft sein die automatische Dereferenzierung zu deaktivieren und dem Nutzer die Verantwortung der Dereferenzierung zu übertragen oder den Nutzer eigenständig entscheiden zu lassen, ob er automatisch dereferenzieren lassen will oder manuell. Die automatische Dereferenzierung kann dabei für eigene Objekte mit dem Überschreiben mehrerer virtueller Funktionen des `IObject` Interfaces konfiguriert werden. Wenn keine uniformisierten

Attribute von Objekten zurückgegeben werden, reicht die im `INativeObject` deklarierte Dereferenzierungslogik aus.

3.2.1.3 Scopes

Die Handhabung und Zugriffsverwaltung von Variablen geschieht über stringbasierte Variablentabellen, sogenannte Scopes. In diesen Scopes werden Variablen unter einem Label abgespeichert. Zudem enthalten die Scopes Zeiger auf den ihnen übergeordneten Scope. So kann aus hierarchisch untergeordneten Kontexten immer auf die globaleren Variablen des übergeordneten Kontextes zugegriffen werden. Bei Verlassen eines Kontextes wird der entsprechende Scope dekonstruiert. Dabei wird die automatische Dereferenzierung durchgeführt und im Anschluss werden alle Variablen gelöscht. Da Alberich keine Referenzzählung implementiert und die automatische Dereferenzierung ein optionales, präventives Feature ist und damit bei performancekritischen Ausführungen einfach abgeschaltet werden kann, basiert die Dereferenzierung der Variablen auf einem rohen, enumerativen Abgleich. Werden externe Referenzen auf lokale Variablen gefunden, werden diese über die Referenzierung der Nullreferenzinstanzen dereferenziert. Algorithmus 5 zeigt den exemplarischen Aufbau der Scope-Speicherstruktur.

Algorithmus 5: Aufbau des Scopes

```
1 typedef std::map<std::string, Variable> VariableTable;
2
3 struct Scope {
4
5     VariableTable variableTable = {};
6     Scope* parent = nullptr;
7
8     ~Scope(){ ... }   // automatische Dereferenzierung
9 };
```

3.2.1.4 Evaluierungsergebnisse

Ein Evaluierungsergebnis ist das Resultat der Auswertung eines Ausdrucks und entspricht entweder einer temporären (RValue) oder einer in einem Scope hinterlegten Variablen (LValue). Da die beiden Ergebnisse unterschiedlich gehandhabt werden müssen, implementiert Alberich eine vereinheitlichende Darstellung. Dabei werden LValues als Zeiger auf den entsprechenden Scopeeintrag weitergegeben und RValues als konkrete Instanz. Die Struktur, die das Evaluierungsergebnis darstellt, enthält dafür ein Zeiger- und ein Instanzattribut. Ob ein Evaluierungsergebnis ein LValue, ein RValue oder ungültig ist, kann anhand der Gültigkeiten des Zeigers und der Instanz identifiziert werden. Algorithmus 6 demonstriert den beschriebenen Aufbau der Evaluierungsergebnisstruktur.

Algorithmus 6: Aufbau des Evaluierungsergebnisses

```
1 struct EvalResult {
2
3     Variable variable;
4     Variable* variablePtr = nullptr;
5
6     bool isLValue() const{ return variablePtr != nullptr; }
7     bool isRValue() const{ return !isLValue(); }
8 };
```

3.2.2 Sprachregister

Dieses Kapitel gibt einen Überblick über die Sprachregister, denen sich der Interpreter während der Evaluierung bedient.

3.2.2.1 Typenregister

Damit registrierte Typen trotz der polymorphen Vereinheitlichung zur Laufzeit unterschieden werden können, werden sie registriert. Die Registrierung erfolgt in einem Typenregister und erfordert die Übergabe eines Keywords für den Objekttypen und einen Standardkonstruktor, der eine polymorphe `IObject` Instanz zurückgibt. Bei der Registrierung wird der übergebenen Objektklasse ein einzigartiger Typenindex zugeordnet. Für diesen Typenindex werden das übergebene Keyword und der Standardkonstruktor in einer Map hinterlegt. Die Zuordnung von Keyword auf Typenindex wird ebenfalls gespeichert. Nach der Registrierung wird der eindeutige Typenindex zurückgegeben. Wenn eine eigene Implementierung der Objektindizierung und ein direkter Aufruf der Typenregistrierungsfunktion genutzt wird, empfiehlt es sich den vergebenen Typenindex in einer statischen oder externen Variablen zu speichern und die gelöschte virtuelle Funktion `TypeIndex IObject::getTypeIndex` mit einer Funktion zu überschreiben, die diesen gespeicherten Typenindex zurückgibt. So wird die Laufzeittypisierung des Objekts trotz polymorpher Vereinheitlichung ermöglicht. Die beschriebene Implementierung kann über die Verwendung der Interfaces `IIndexedObject` und `INativeObject` automatisiert werden. Algorithmus 7 zeigt eine exemplarische Deklaration des Typenregisters.

Algorithmus 7: Aufbau des Typenregisters

```
1  struct TypeRegister{
2
3    TypeIndex typeCounter;
4
5    // Zuordnung Typenindex -> TypeInfo(struct mit Keyword, Standardkonstruktor)
6    std::map<TypeIndex, TypeInfo> typeInfos = {};
7
8    // Zuordnung Keyword -> Typenindex
9    std::map<std::string, TypeIndex> typeIndices = {};
10
11   TypeIndex registerType(const std::string& keyword, const std::function<IObject*()>&
     ↪  initConstructor){
12
13     typeInfos.emplace(typeCounter, TypeInfo{keyword, initConstructor, typeCounter});
14     typeIndices.emplace(keyword, typeCounter);
15
16     return typeCounter++;
17   }
18 };
```

3.2.2.2 Keywordregister

Um konstante Objekte zur Verfügung zu stellen, die nicht manipuliert und nicht referenziert werden können, wird ein Keywordregister implementiert. In diesem Register werden konkrete Objekte für Keywords gespeichert. Die Implementierung von Objekten in das Register dient dem kontextunabhängigen Bereitstellen von Konstanten unter einem Label. Damit werden Objekte freigeschaltet, ohne dass die Sprachdistribution eigens hierfür Variablen in den übergeordneten

Interpreterscope implementieren muss. Zudem sind die Konstanten so unveränderbar und unüberschreibar. Alberich registriert eigenständig Konstanten für die Keywords `true`, `false` und `nullArgs`.

3.2.2.3 Funktionsregister

Zur Freischaltung von Backendfunktionen über die Sprache werden Funktionsregister implementiert. Funktionsregister sind Speicherstrukturen, die Funktionen abspeichern, verwalten und aufrufen. Dabei muss die Funktion in einem einheitlichen Format hinterlegt werden. Die Parameter des einheitlichen Funktionsformats sind dabei die Objektinstanz, vom Interpreter evaluierte Eingaben, die Typenindizes dieser Eingaben, der Container, in den die Rückgaben der Funktion geschrieben werden und der Scope, in den die Funktion zurückgegeben wird. Die Objektinstanz ist dabei nur ein gültiger Zeiger, wenn die Funktion, die aufgerufen wird eine Attributsfunktion einer Objektinstanz ist. Algorithmus 8 demonstriert dieses vereinheitliche Funktionsformat exemplarisch.

Algorithmus 8: Von Funktionsregister erwartetes Funktionsformat

```
1 typedef std::vector<EvalResult>& FunctionReturns;
2 typedef std::vector<EvalResult*> FunctionParams;
3 typedef EvalResult* TypeMember;
4
5 #define FREG_ARG_TYPES \
6     FunctionReturns, const FunctionParams&, Scope&, const std::vector<TypeIndex>&, TypeMember
```

Funktionsregister ermöglichen das Abspeichern von Funktion dieses Formats. Dabei wird die in das Registerformat überführte Funktion unter dem Funktionsnamen und der Typenindizes der erwarteten Inputparameter abgespeichert. Damit ist das Register für Funktionen mit gleichen Namen und verschiedenen Inputparametern überladbar. So können Funktionen mit gleichem Namen in das Register eingelagert werden, solange sich die Typenindizes der Inputparameter unterscheiden. Über Angabe des Namens und der Inputindizes kann eine Funktion im Register lokalisiert und aufgerufen werden. Für den Aufruf werden die Inputparameter des Funktionsformats und ein Rückgabecontainer vom Interpreter bereitgestellt. Wird für die vorliegenden Inputparameter keine passende Überladung gefunden überprüft der Interpreter zusätzlich, ob eine Überladung für beliebige Inputparameter existiert.
Algorithmus 9 zeigt diesen exemplarischen Aufbau des Funktionsregisters

Algorithmus 9: Aufbau des Funktionsregisters

```
1 typedef std::function<void(FREG_ARG_TYPES)> IObjectFunction;
2 typedef std::pair<std::string, std::vector<TypeIndex>> FunctionRegisterKey;
3 typedef std::map<FunctionRegisterKey, IObjectFunction> FunctionMap;
4
5 class FunctionRegister {
6
7   FunctionMap functions;
8
9   void registerFunction(const std::string& functionLabel, const std::vector<TypeIndex>&
    ↪  functionArgsTypes, const IObjectFunction& func, ...) {
10
11    functions[{functionLabel, functionArgsTypes}] = {func,functionReturnTypes};
12  }
13
14  void callFunction(const std::string& functionLabel, FREG_DEFAULT_CALLARGS_WITH_MEMBER) {
15
```

```
16     auto fIt = functions.find({functionLabel, getArgTypes(functionParams)});
17
18     if(fIt != functions.end()){
19
20       fIt->second.first(returns, functionParams, returnToScope, fIt->second.second, member);
21     }
22   }
23 };
```

3.2.2.4 Backendregister

Typen- und Funktionsregister ermöglichen durch die Registrierungen von Objekten, eigenständigen, statischen und Attributsfunktionen die vollständige Freischaltung eines Backends. Um Freischaltungen modular und getrennt voneinander bereitstellen zu können, implementiert Alberich ein Backendregister. Dieses Register steuert und verwaltet die Freischaltung verschiedener Backends. Dabei speichert das Register Zeiger auf Funktionen, die die Backendfreischaltungen vornehmen unter dem entsprechenden Backendlabel ab und ruft die Freischaltungen auf Anfrage gezielt auf. So können Freischaltungen abrufbar bereitgestellt werden, ohne direkt jede Funktionalität in die Register zu implementieren und diese dadurch aufzublähen. Die Entscheidung darüber welche bereitgestellten Freischaltungen aufgerufen werden unterliegen so direkt dem Nutzer.

3.2.2.5 Registerinstanzen

Die beschriebenen Register werden für die Nutzung global instanziert und der Aufruf der Register über mehrere Wrapperfunktionen gepackt. Dabei kommen mehrere Funktionsregister zum Einsatz. Die eigenständigen Funktionen werden in einem globalen Funktionsregister gespeichert. Für jeden freigeschalteten Typen werden Funktionsregister unter dem eindeutigen Typenindex in Maps für die statischen und Attributsfunktionen gespeichert.

3.2.3 Interpreter

In diesem Kapitel wird die Implementierung des Alberich-Interpreters beschrieben. Dieser besteht aus vier zentralen Komponenten.

Komponenten des Interpreters

- Präprozessor : Vorverarbeitung
- Lexer : lexikalische Analyse
- Parser : syntaktische Analyse
- Evaluierer : semantische Analyse

Diese zentralen Komponenten bilden eine Verarbeitungskette, die das Skript des Nutzers entgegennimmt, übersetzt und ausführt. Der Interpreter gibt die Ergebnisse der Evaluierung als Ausgaben oder Ergebnisdateien an den Nutzer zurück. Die konkrete Interpreterkomponente arbeitet dabei immer mit dem Output der vorherigen. Der Output der Komponente wird wiederum an die nachgestellte Komponente weitergereicht. Dabei ist der Input der ersten Komponente das Skript, der Output der letzten Komponente die an den Nutzer zurückgeführten Ergebnisse. Der

Präprozessor nimmt dabei das Skript entgegen, formatiert es um und gibt das umformatierte Skript als String zurück. Dieser wird an den Lexer weitergereicht, der den String lexikalisch analysiert und in eine Tokenfolge aufteilt. Diese Folge wird durch den Parser im Rahmen der syntaktischen Analyse in einen Syntaxbaum überführt. Der Evaluierer ist für die semantische Analyse oder Evaluierung des hergestellten Syntaxbaums zuständig, interpretiert diesen und führt so die übersetzten Anweisungen des Nutzers aus. Er ist für die Ausgaben und Herstellung der Ergebnisdateien zuständig, die an den Nutzer zurückgeführt werden. Abbildung 4 stellt diesen Verarbeitungsaufbau exemplarisch dar.

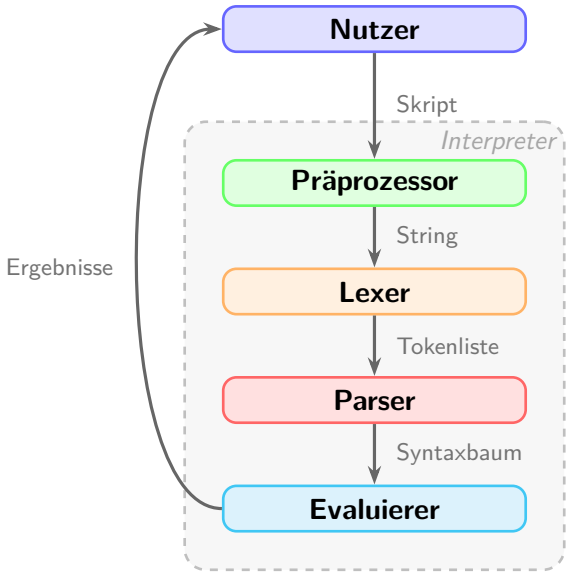


Abbildung 4: Verarbeitungskette des Interpreters

3.2.3.1 Präprozessor

Der Präprozessor ist die Interpreterkomponente, die das unverarbeitete Skript übergeben bekommt. Er formatiert das Skript um und gibt einen String zurück, der durch den Lexer weiterverarbeitet wird. Die konkrete Steuerung erfolgt dabei durch Präprozessordirektiven oder sogenannte Makros. Alberichs Präprozessor beschränkt sich dabei allerdings auf eine Umformatierung der Skripte, die das Schreiben der Sprache angenehmer und die Syntax weniger sperrig macht. Dabei werden an den Zeilenenden automatisch Trennzeichen eingefügt, die die Zeilen im Ausgabestring voneinander separieren. Diese Formatierung kann durch die Angabe der Direktive \ am Zeilenende für die konkrete Zeile ausgelassen werden.

3.2.3.2 Lexer

Der Lexer übernimmt die lexikalische Analyse des Skripts. Dabei bekommt er das formatierte Skript als String übergeben und teilt es in einen Tokenvektor auf. Der Tokenvektor wird nach der lexikalischen Analyse an den Parser übergeben. Ein Token enthält dabei die Information, was für eine Funktion es erfüllt, wie Argument, Konstante oder Operator, auf welcher Hierarchieebene es agiert, an welcher Stelle in der Ausgangszeichenfolge, also dem Skript es sich befindet und wie viel Platz es dort einnimmt. Der Lexer, den Alberich implementiert, erfüllt dabei eine Besonderheit. Der Operatorensatz ist vollständig konfigurierbar. Klassische Lexer teilen nach einem vorgegebenen Satz an Operatoren. Da der Alberich-Lexer jedoch für das Le-

xen einer tensoralgebraischen Syntax genutzt werden und der Distribution die Möglichkeit geben soll, die Operatorenauswahl dynamisch anzupassen und zu überladen, wird er dynamisch konfigurierbar gestaltet. Dazu wird der Operatorensatz des Lexers als Stringvektor übergeben. Die Hierarchieebene des Tokens beeinflussende Elemente, wie runde, eckige oder geschweifte Klammern, können nicht konfiguriert oder dynamisch implementiert werden. Algorithmus 10 zeigt eine exemplarische Deklaration des Tokenobjekts.

Algorithmus 10: Aufbau des Lexer-Tokens

```
1 enum class TkType : uint8_t {
2
3   None,               // default
4   Argument,           // Substitutionsargument
5   Constant,           // ausgeschriebene Zahl
6   Operator,           // Operator
7   ...
8 };
9
10 struct Token{
11
12   TkType type;
13   size_t hierarchie;
14   size_t position, length;
15
16   // fallspezifische Attribute
17   ...
18 };
```

3.2.3.3 Parser

Der Parser führt die syntaktische Analyse durch. Als Input bekommt er die Tokenfolge, die der Lexer erzeugt hat und überführt sie in eine abstrakte Baumstruktur (AST). Diese Struktur ist aus einzelnen Knotenelementen aufgebaut. Die einzelnen Knoten sind dabei Instanzen einer rekursiven Speicherstruktur. Das Knotenobjekt enthält Informationen darüber, welche Funktion es einnimmt, welche Tokens es umfasst, und welche untergeordneten Knoten es verknüpft. Dabei enthält das Knotenobjekt diese untergeordneten Knoten oder Abhängigkeiten direkt. Ein konventioneller AST (binäre Konvention) hat dabei maximal zwei Abhängigkeiten. Alberich hingegen nutzt zur effizienteren Evaluierung die n-äre AST-Konvention. Diese erlaubt die Verknüpfung beliebig vieler Abhängigkeiten durch einen AST-Knoten. Der Algorithmus 11 zeigt die exemplarische Struktur eines Knotenobjekts.

Algorithmus 11: Aufbau des Parser-AST-Knotens

```
1 struct ASTNode{
2
3   TkType Relation = TkType::None;
4
5   size_t begin = 0, end = 0;          // Position in Tokenfolge
6   size_t depth = 0;                   // Rekursionstiefe
7
8   std::vector<ASTNode> children = {}; // Verknüpfte Nodes
9 };
```

Die dynamische Konfigurierung des Parsers erfolgt durch die Angabe der Operatorenhierarchie. Diese wird ebenfalls durch den Operatorensatz eingestellt, der für die Einrichtung des Lexers an

den Interpreter übergeben wird. Die Hierarchie eines Operators entspricht dabei seiner Position in dieser Folge. Steht ein Operator in dieser Folge weiter vorn als ein anderer, wird er gegenüber diesem präferiert. Das bedeutet, dass der präferierte Operator innerhalb der selben Hierarchiebene als makroskopischere Operation erkannt wird als der nicht präferierte. Deshalb wird dieser bei der rekursiven Verknüpfung der Tokenfolge bevorzugt. Soll zum Beispiel Punkt-vor-Strich-Rechnung korrekt implementiert werden müssen die Operatoren `+` und `-` in der übergebenen Operatorenfolge weiter vorn stehen als die Operatoren `*` und `/`. Damit wird sichergestellt, dass bei der syntaktischen Analyse die Strichrechnung als makroskopischere Operation erkannt wird und Punktrechnungen als Abhängigkeiten eingelagert werden. Da die Hierarchie der Operatoren durch eine Position innerhalb der Operatorenliste übertragen wird, können Operatoren nicht auf dem gleichen Präferenzlevel operieren. Das bedeutet, dass Klammern in einigen Fällen expliziter gesetzt werden müssen, als in anderen Sprachen, die Operatoren mit gleicher Präferenz ermöglichen. Zudem stuft der Parser den Kommaoperator innerhalb von runden Klammern als dominanteste Operation ein. Das soll gewährleisten, dass unabhängig von der Distributionskonfiguration immer ein Container existiert, der die Unterteilung nach dem Kommaoperator anstrebt. Alberich nutzt dabei standardmäßig eine abweichende Hierarchisierung des Kommaoperators. Durch das Setzen der runden Klammern kann jedoch immer eine Unterteilung nach dem Kommaoperator forciert werden. Der Algorithmus 12 zeigt auf, wie verschiedene Parserkonfigurationen zu unterschiedlichen Ausdrucksauswertungen führen.

Algorithmus 12: Einfluss der Operatorenhierarchie auf die Evaluierung

```
1 configOperands({"+", "-", "*", "/"});
2 eval("2 + 3 * 4")
3 // -> wird interpretiert als "2 + (3 * 4)"
4
5 configOperands({"*", "/", "+", "-"});
6 eval("2 + 3 * 4")
7 // -> wird interpretiert als "(2 + 3) * 4"
```

3.2.3.4 Evaluierer

Der zentrale Kern des Alberich Interpreters ist der Evaluierer. Er erhält den abstrakten Syntaxbaum, den der Parser herstellt, als Eingabe, evaluiert ihn und führt dabei die evaluierten Anweisungen aus. Für eine solche Evaluierung eignen sich mehrere Algorithmen. Die Auswahl eines Evaluierungsalgorithmuses sollte dabei die Zielanwendung der Sprache und den einhergehenden Implementierungsaufwand des Algorithmuses berücksichtigen. Der Alberich- oder Distributions-Interpreter erwartet nur verhältnismäßig kurze Steuer- und Verwaltungsskripte. Die Implementierung großer Bibliotheken und performancekritischer Ressourcen ist über das Freischalten eines C++ Backends in jedem Fall effizienter als über eine Implementierung in der hergestellten Interpretersprache. Der Interpreter ist damit nicht auf eine optimierte Evaluierung angewiesen. Die Stärke der Sprache liegt in der Nutzung der performanten Backendobjekte und -routinen. Aus diesem Grund reicht eine rekursive Evaluierung mit stringbasierten Scopes aus. Eine VM- (virtuelle Maschine) oder stackbasierte Evaluierung ist zwar effizienter, der Unterschied für den vorgesehenen Anwendungsfall jedoch vernachlässigbar. Eine Implementierung dieser Algorithmen hingegen bedeutet einen enormen Mehraufwand. Der Overhead, der durch die rekursive Evaluierung entsteht, skaliert dabei linear mit der Komplexität des evaluierten Syntaxbaums. Da die eigentlichen Operationen durch die freigeschalteten, nativen Ressourcen geleistet werden, bleibt die Ausführung trotzdem auch für umfangreichere Skripte und trotz eines größeren Evaluierungsoverheads praktisch noch effizient.

3.2.3.4 Rückgabe

Die Rückgabe der rekursiven Evaluierungsroutine besteht aus einer Evaluierungsergebnisliste und einem Abbruchsfall. Das ermöglicht das parallele Arbeiten mit mehreren Objekten pro iterativem Evaluierungsschritt und eine erweiterte Steuerung des Abbruchs der Evaluierung, wie durch Return-, Break- oder Continuestatements. Der Aufbau dieses Rückgabetypens wird in Algorithmus 13 demonstriert.

Algorithmus 13: Aufbau des ProcessingResults

```
enum class ExitCase : uint8_t{ None, Continue, Break, Return, };

struct ProcessingResult{

  ExitCase exit = ExitCase::None;
  std::vector<EvalResult> evalResults = {};
}
```

3.2.3.4 Evaluierungsroutine

Die zentrale Evaluierungsroutine verarbeitet den abstrakten Syntaxbaum, der vom Parser erstellt worden ist rekursiv. Dabei ermittelt der einzelne Evaluierungsschritt den vorliegenden Anwendungsfall und gibt das Ergebnis einer fallspezifischen Verknüpfung der rekursiv evaluierten Knotenabhängigkeiten zurück. Algorithmus 14 zeigt diese Grundstruktur exemplarisch.

Algorithmus 14: Funktionsweise der rekursiven Evaluierungsroutine

```
ProcessingResult evaluateExpression(const ASTNode& node, ...){

  ProcessingResult prcResult;

  // Ermitteln des vorliegenden Anwendungsfalls
  switch(node.Relation){

    // fallspezifische Verknüpfung der evaluierten Abhängigkeiten
    prcResult = \
      f(evaluateExpression(node.children[0], ...), ..., evaluateExpression(node.children[n], ...))
  }

  return prcResult;
};
```

Dabei ruft die Evaluierungsroutine sich so lange rekursiv selbst auf, bis ein Knoten evaluiert wird, dessen Anwendungsfall keine weitere rekursive Evaluierung erfordert. In den meisten Fällen sind das externe Knoten, die damit keine Abhängigkeiten enthalten.

externe AST Knoten

- Ausgeschriebene Zahlen
- Ausgeschriebene Strings
- Variablennamen
- Registrierte Keywords (im Register hinterlegte Konstanten)
- Andere Keywords (return, break, decl, ...)

Ausgeschriebene Zahlen und Zeichenketten werden dabei automatisch für die Konstruktion von Double-, Int- oder String-Objekten genutzt. Diese Typen schaltet Alberich standardmäßig selbst frei. Dabei werden die entsprechenden Objekte direkt als RValues in die zurückgegebene Größe konstruiert. Die Evaluierung des Namens einer registrierten Variablen führt zur Konstruktion eines LValues, die eines registrierten Keywords zur Konstruktion eines RValues. Dieser RValue enthält dabei eine Kopie des im Keywordregister hinterlegten Objekts. So wird gewährleistet, dass für Keywords hinterlegte Objekte durch den Nutzer nicht geändert werden können.

3.2.3.4 Überladungskonfiguration

Die Ergebnisse dieser Evaluierungen externer Knoten werden in andere Anwendungsfälle zurückgegeben und dort als Abhängigkeiten verknüpft. Das geschieht zum Beispiel über Operatoren. Da Alberich mit einem unkonventionellen (n-ären) AST arbeitet, können dabei einzelne Knoten beliebig viele Abhängigkeiten verknüpfen. Zur besseren Unterscheidbarkeit werden hier mehrere Anwendungsfälle behandelt, in denen Operatoren zum Einsatz kommen.

Mögliche Operatoranwendungen

- Unär, $-x$
- Binär, $y - x$
- Verkettung, $y - x - ...$

Für jeden dieser Anwendungsfälle können Operatoren überladen werden. Überladung bezeichnet dabei das Hinterlegen einer Funktion für einen Anwendungsfall und einen Operator, die für die Verknüpfung von Abhängigkeiten über den konkreten Operator im konkreten Anwendungsfall aufgerufen wird. Für die Überladung einer unären Operatorenverwendung wird eine Funktion mit einem Input hinterlegt, für die binäre und die verkettende Verwendung eine Funktion mit zwei Inputparametern. Dabei darf die Funktion, die die Verwendung des Operators im verkettenden Zustand definiert, nichts zurückgeben. Das liegt daran, dass die Verknüpfung einer Operandenkette in Alberich eine einmalige Kopie des ersten Operanden vorsieht, mit der weitergearbeitet wird. RValues werden dabei nicht kopiert, sondern direkt verschoben, trotzdem wird im Weiteren von Kopie gesprochen. Die angegebene Überladungsfunktion für die verkettende Anwendung operiert dann direkt auf dem ersten übergebenen Inputparameter. Das ist im Fall der Evaluierung einer Operandenkette die Kopie des ersten Operanden. Die verkettende Überladungsfunktion wird dann in Schleife jeweils mit der temporären Kopie des ersten Operanden und dem zweiten bis letzten Operanden aufgerufen. Dabei wird die temporäre Kopie des ersten Operanden innerhalb jeden Aufrufs der Überladungsfunktion vor Ort manipuliert, bis er dem Ergebnis der Operandenverknüpfung entspricht. Diese Behandlung ermöglicht die Evaluierung beliebig langer Operandenketten mit nur einem Kopiervorgang. Algorithmus 15 zeigt diese Verknüpfung exemplarisch.

Algorithmus 15: Exemplarische Verknüpfung einer Operandenkette

```
ProcessingResult evaluateExpression(const ASTNode& node, ...){

  ...

  if(IsOperandsChain(node.Operator)){

    // Zuweisung kopiert hier LValues und verschiebt RValues
    Evalresult tmp = evaluateExpression(node.children[0], ...).evalResults[0];

    for(size_t childIdx = 1; childIdx < node.children.size(); childIdx++){

      callFunction(..., {tmp, evaluateExpression(node.children[childIdx], ...).evalResults[0]});
    }

    prcResult.append(tmp)
  }

  ...
};
```

Wird ein Operator dabei nicht für die binäre Verwendung überladen, wird bei einer binären Verwendung die Überladung für die Verkettung angewendet. Diese Angabe der Überladung geschieht dabei über drei Maps, die Alberich implementiert. Die Distribution stellt die Überladungskonfiguration über diese drei Maps ein. Dabei kann die Möglichkeit der Überladungskonfiguration auch direkt dem Nutzer ermöglicht werden. Algorithmus 16 zeigt die exemplarische Angabe der Überladungskonfiguration für den Minusoperator.

Algorithmus 16: Angabe der Überladungskonfiguration

```
g_OneArgOperations = {{"-", "__negate__"}};
g_TwoArgOperations = {{"-=", "__subAssign__"}};
g_ArgChainOperations = {{"-", "__subAssign__"}};

// eval("-4") ruft __negate__(4) auf
// eval("x -= y") ruft __subAssign__(x, y) auf
// eval("x - y") ruft __subAssign__(copiedX, y) auf
```

3.2.3.4 Steuerstatements

Für die Implementierung von Steuerstatements wie `return`, `break` oder `continue` existiert das `ExitCase`-Attribut der Funktionsrückgabe. Wird ein AST-Knoten evaluiert, der ein Steuerstatement ist, wird der Abbruchsfall der Rückgabe auf den fallspezifischen Wert ungleich des Defaultwertes gesetzt. Der Knoten, der das Steuerstatement als Abhängigkeit evaluiert, checkt den Abbruchsfall der evaluierten Abhängigkeit und fährt nur mit der Standardbehandlung des Anwendungsfalls fort, wenn der Abbruchsfall der Defaultwert ist. Andernfalls wird die Evaluierung abgebrochen und die Rückgabe der Abhängigkeit mit gesetztem Abbruchsfall vorzeitig zurückgegeben. Das geschieht so lange, bis das steuerstatementspezifische Abbruchkriterium erreicht ist. Im Falle eines `break`-Statements ist das beispielsweise das Verlassen des Schleifenscopes, in dem das Statement abgesetzt worden ist. Ein `return`-Statement hingegen sorgt so lange für einen vorzeitigen Abbruch der Evaluierung, bis die evaluierten Ergebnisse, die über die Rückgabe durchgereicht werden, den Kontext erreichen, aus dem die Funktion, die das `return` Statement absetzt, aufgerufen worden ist. So kann über verschiedene Statements der Abbruch von Evaluierungen gezielt gesteuert werden.

3.2.3.4 Kontexte

Für die Evaluierung weiter Anwendungsfälle bedient sich die Evaluierungsroutine eines Kontextparameters. Da aus dem rekursiven Evaluierungsschritt heraus häufig nicht erkennbar ist, aus welcher makroskopischen Umgebung die Evaluierung der Abhängigkeit aufgerufen worden ist, muss diese Information über einen in der Rekursion mitgereichten Parameter mitgeteilt werden. Zudem unterscheidet sich die Behandlung eines Anwendungsfalls je nach Kontext. Wird beispielsweise eine Funktionsdeklarierung ohne makroskopisch gesetzten Kontext evaluiert, wird sie in das Register für die unabhängigen Funktionen eingetragen. Wird die Deklarierung aber innerhalb einer Klassendefinition angestellt, handelt es sich dabei um die Registrierung einer statischen oder Attributsfunktion. Der Kontextparameter ist dabei eine Enum-Instanz. Weitere Informationen, wie der Typenindex der aktuell evaluierten Klassendefinition, werden in externen, fallspezifischen Variablen gespeichert. Algorithmus 17 demonstriert diese Kontextmitteilung innerhalb der Evaluierungsroutine.

Algorithmus 17: Kontextevaluierung und -mitteilung in der Evaluierungsroutine

```
1  enum class Context : uint8_t{ NONE, ASSIGN_LEFTSIDE, ASSIGN_RIGHTSIDE, DECL_STRUCT, ... };
2
3  ProcessingResult evaluateExpression(const ASTNode& node, ..., Context context){
4
5    ...
6
7    if(isStructDecl(node)){ evaluateExpression(..., Context::DECL_STRUCT); }
8    else if(isFuncDecl(node) && context == Context::DECL_STRUCT){
9      /* Registrierung klassenspezifischer Funktion */
10   }
11   else if(isFuncDecl(node)){ /* Registrierung Unabhängiger Funktion */ }
12
13   ...
14 }
```

3.2.3.4 Fehlerbehandlung

Bei Start der Evaluierung wird eine Funktion, die Informationen über den aktuellen Fehlerkontext schreibt an die Assertions angeschlossen mit denen Alberich arbeitet. Es empfiehlt sich deshalb die gleichen Assertions im Backend zu nutzen. Die Kontextfunktion kann dabei auch an eigene Routinen angeschlossen werden. Wird eine solche Assertion ausgelöst, erzeugt sie über den Aufruf der Fehlerkontextfunktion einen Kontext-String und schreibt diesen als erweiterten Fehlerkontext hinter die Fehlermeldung, die vom Programm erzeugt wird. Der String enthält dabei alle Zeilen des evaluierten Skriptes, über die sich der Knoten erstreckt, bei dessen Evaluierung ein Fehler auftritt. Dabei wird der String direkt mit Zeilennummern formatiert und der Bereich, über den sich der Knoten erstreckt, unterstrichen. Um diesen Fehlerkontext bereitstellen zu können, implementiert und setzt der Evaluierer bei jeder rekursiven Ausführung der Evaluierungsroutine einen global gültigen Zeiger auf den aktuell evaluierten Knoten. Nachdem der rekursive Evaluierungsschritt abgeschlossen ist, wird dieser Zeiger auf seinen ursprünglichen Wert zurückgesetzt. Wird eine Assertion ausgelöst, ist der Knoten, der diesen Fehler verursacht, der in dem Zeiger hinterlegte. Dadurch kann für jede Fehlermeldung, die das freigeschaltete Backend oder der Interpreter selbst absetzt, ein skriptbezogener Kontext mitgeliefert werden. Algorithmus 18 zeigt den Anschluss der Fehlerkontextfunktion innerhalb eines beispielhaften Assertion-Makros.

Algorithmus 18: Anschließen der Fehlerkontextfunktion

```
// Automatischer Anschluss der Funktion bei Start des Evaluierers
std::string (*g_getErrorContext)() = nullptr;
g_getErrorContext = &getErrorContex

// Einbindung in eine Präprozessordirektive, Anschluss an eigene Fehlerausgabe
#define ASSERT(condition, message)\
  do{\
      if (!(condition)) {\
          \
          std::ostringstream oss; oss << ...; \
          if(g_getErrorContext != nullptr){ oss << g_getErrorContext(); } \
          _ERROR << oss.str() << ENDL;\
      }}\
  while(0);
```

3.3 Baldur

In diesem Kapitel wird die Implementierung der Sprache Baldur vorgestellt. Dabei wird in erster Linie das Tensoralgebra Backend beschrieben. Eine Freischaltungsanleitung wird in Kapitel 4.1.1 beschrieben.

3.3.1 Überladungskonfiguration

Baldur implementiert eine eigene Basisüberladungskonfiguration. Diese kann allerdings dynamisch durch den Nutzer angepasst oder erweitert werden. Dazu wird die Datei `__LPECONFIG.JSON` verwendet. Sie übergibt die Überladungskonfiguration im gut verständlichen JSON-Format. Existiert eine solche Datei im gleichen Verzeichnis wie das auszuführende Skript, wird die Überladungskonfiguration aus dem Skript eingelesen und angewendet. Fehlt die entsprechende Datei wird sie mithilfe der implementierten Basisüberladungskonfiguration erzeugt. Dabei verwendet die Datei die Schlüssel `LEXICON`, `PREFIX`, `INFIX` und `FOLD`. Für diese Schlüssel werden die Operatorenauswahl nach Hierarchie, die unären Überladungen, die binären Überladungen und die verkettenden Überladungen angegeben. Für die Überladung eines Operators ist es damit nicht nur notwendig, diesen unter dem entsprechenden Schlüssel mit der dazugehörigen Überladungsfunktion anzugeben, sondern auch an der richtigen Stelle in die Operatorenhierarchie einzutragen. Um Fehlern vorzubeugen, wird vor Anwendung der Konfiguration eine Konsistenzprüfung durchgeführt. Wird dabei ein Operator überladen, der nicht in der Operatorenauswahl angeführt wird, kommt es zu einer Fehlermeldung.

3.3.2 Basistypen

Die zentrale Zielanforderung an die Sprache Baldur ist das Arbeiten mit algebraischen und indexnotierten Tensorausdrücken. Dazu werden mehrere Basistypen implementiert und freigeschaltet, die diese Ausdrücke symbolisch repräsentieren. Symbolische Darstellung bezeichnet hier das Phänomen, dass die Ausdrücke durch Objekte repräsentiert werden, die Informationen über Ausdrucksstruktur und -aufbau beinhalten, aber keine konkreten Werte für Tensoreinträge. Die Darstellung wird über eine abstrakte Syntaxbaumstruktur realisiert. Die Knoten der Baumstruktur sind rekursive Speicherstrukturen, die andere Knotenobjekte enthalten. Von einem Knoten enthaltenen Kindknoten werden als dessen Abhängigkeiten bezeichnet. Ein Knotenobjekt speichert dabei den Knotentyp, knotentypspezifische Details, die Tensorstufe, die Tensordimensio-

nen sowie dessen Abhängigkeiten. Der Knotentyp unterscheidet dabei zwischen externen Knoten ohne Abhängigkeiten und Operationsknoten, die andere Knotenobjekte verknüpfen. Für diese spezifischen Knotentypen werden zusätzliche Informationen gespeichert, wie der verknüpfende Operator und das Label des externen Knotens. Die Berücksichtigung der Tensordimensionen ist dabei optional. Werden bei der Konstruktion des algebraischen Ausdrucks keine Tensordimensionen angegeben, bleiben sie bei dessen weiterer Behandlung unberücksichtigt. In den Knoten der indexnotierten Ausdrücke werden zusätzlich die dem Ausdruck zugewiesenen Indizes gespeichert. Algorithmus 19 zeigt die exemplarische Definition des Knotenobjekts zur symbolischen Darstellung algebraischer Tensorausdrücke.

Algorithmus 19: Aufbau der symbolischen Darstellung algebraischer Tensorausdrücke

```
1  struct TensorExpression{
2
3    TkType Relation = TkType::None,               // Knotentyp
4
5    TensorExpressionOperator Operator = ...;      // Knotentypspezifische Informationen
6    std::string label = NULLSTR;                  //
7
8    int tensorOrder = -1;                         // Algemeine Informationen über den Ausdruck
9    std::vector<int> dimensions = {};             //
10
11   std::vector<TensorExpression> children = {};  // Knotenabhängigkeiten
12 };
```

3.3.3 Konzipierter Ablauf

Die algebraischen Ausdrücke sind ein eigenständiges Format und werden dem Nutzer nahezu vollständig freigeschaltet. Die indexnotierten Ausdrücke sind kein eigenständiges Format. Sie dienen in erster Linie als Exportformat der algebraischen Ausdrücke. Das bedeutet, dass die algebraischen Methoden der indexnotierten Ausdrücke nur teilweise freigeschaltet werden und in erster Linie für eine backendinterne Nutzung konzipiert sind. Der vorgesehene Arbeitsfluss ist das Arbeiten mit algebraischen Ausdrücken bis zum Ergebnis. Nach der Ermittlung der Ergebnisgröße wird der algebraische Ausdruck in Indexnotation umgewandelt. Der indexnotierte Ausdruck wird dann in programmexterne Formate exportiert.

3.3.4 Ausdruckskonventionen

Um dem Nutzer eine höhere Kontrolle über die Ausdrucksstruktur zu bieten, implementiert Baldur zwei AST Konventionen, nach denen die algebraischen und indexnotierten Ausdrücke gebaut werden können. Dabei handelt es sich um die binäre und die n-äre Konvention. Knotenobjekte der binären Konvention können dabei maximal zwei Kindknoten enthalten, Knotenobjekte der n-ären Konvention hingegen beliebig viele. Dabei werden Ausdrücke der binären Konvention als gepackte Ausdrücke und Ausdrücke der n-ären Konvention als entpackte Ausdrücke bezeichnet. Die Kontrolle darüber, welche Konvention genutzt werden soll, erfolgt über das Setzen der globalen backendspezifischen Flag `unwrapOperands`.

3.3.5 Algebraische Operationen

Der Bau von algebraischen Ausdrücken erfolgt über die Verknüpfung algebraischer Ausdrücke durch algebraische Operationen. Baldur implementiert dazu einen Satz an algebraischen Ba-

ckendfunktionen. Bei den Funktionen handelt es sich um Attributsfunktionen der Knotenobjekte, die ein weiteres Knotenobjekt als Operanden entgegennehmen. Die Funktion modifiziert die aktuelle Instanz dabei vor Ort. Damit sind die Operanden der algebraischen Operation die angesprochene und die übergebene Instanz. Das Ergebnis der Operation ist die angesprochene Instanz nach Modifizierung durch den Funktionsaufruf. Dabei werden je nach gewählter Ausdruckskonvention Abhängigkeiten gleicher Operationen ge- oder entpackt. Im Gegensatz zu einer Rückgabe der Funktion sorgt die Modifizierung der aufgerufenen Instanz dafür, dass beim Aufbau des Syntaxbaums die Anzahl der benötigten Kopiervorgänge minimiert wird. Algorithmus 20 zeigt die exemplarische Deklarierung der symbolischen, tensoralgebraischen Addition. Dabei wird zunächst sichergestellt, dass die Tensorstufen und -dimensionen der Operanden übereinstimmen. Ist der aktuelle Knoten bereits eine Addition und die gewählte Ausdruckskonvention die entpackte, kommt es zum direkten Anhängen des anderen Knotens an die Abhängigkeiten. Wenn der aktuelle Knoten keine Addition ist oder die gepackte Konvention durchgesetzt werden soll, kommt es vorher zu einer Verschiebung des aktuellen Knotens in die eigene erste Abhängigkeit. Die Verschiebung sorgt dabei dafür, dass zur Identifikation des Knotens relevante Attribute neu gesetzt werden müssen. Deshalb werden einige Attribute wie die Tensorstufe, die Tensordimensionen, der Knotentyp und -operator aus der ersten Abhängigkeit kopiert. Dann wird der übergebene Operand als zweite Abhängigkeit angehängt. Im Anschluss an die aufgezeigte Unterscheidung werden Tensorstufe und Tensordimensionen korrigiert. Die im Knoten hinterlegten Informationen entsprechen immer noch denen des ersten Operanden. Unter Berücksichtigung des übergebenen zweiten Operanden und der stattfindenden Operation werden Tensorstufe und -dimension auf die des erwarteten Operationsresultats angepasst. Für die Tensoraddition ist keine Anpassung nötig, da die Stufe und die Dimensionen des Ergebnisses denen des ersten Operanden entsprechen. Die Implementierungen algebraischer Operationen für indexnotierte Ausdrücke berücksichtigen die Tensordimensionen nicht. Beim Nachbau der algebraischen Ausdrücke in Indexnotation werden die Dimensionen falls berücksichtigt kopiert. Zusätzlich berücksichtigen die Tensoralgebrafunktionen der indexnotierten Ausdrücke die Indizes der Operanden.

Algorithmus 20: Funktionsweise algebraischer Operationsimplementierungen

```
1  void TensorExpression::addAssign(const TensorExpression& other){
2
3    static TensorExpressionOperator operation = TensorExpressionOperator::Addition;
4
5    // Sicherstellen der Stufen- und Dimensionsgleichheit
6    RETURNING_ASSERT(tensorOrder == other.tensorOrder && dimensions == other.dimensions, ...,);
7
8    // Wenn aktueller Knoten keine Addition ist oder gepackte Konvention durchgesetzt werden soll
9    if(!unwrapOperands || Operator != operation){
10
11     // Verschieben des aktuellen Objekts in erste Abhängigkeit
12     moveSelfIntoFirstChild();
13
14     // Objekt als Addition markieren und Stufe und Dimensionen übertragen
15     Operator = operation;
16     tensorOrder = children.back().tensorOrder;
17     ...
18   }
19
20   // Anhängen des anderen Operanden an Abhängigkeiten
21   children.emplace_back(other);
22   const TensorExpression& thisMember = children[children.size() - 2];
23   const TensorExpression& otherMember = children.back();
```

```
24
25   // Dimensionen und Tensorstufe anhand der Operanden setzen
26   // tensorOrder, dimension = f(thisMember, otherMember)
27
28   // Hier aufgrund der Operation (Addition) nicht notwendig
29 }
```

3.3.6 Ausdrucksassemblierung

Durch Verwendung der algebraischen Funktionen lassen sich algebraische Ausdrücke schrittweise aus externen Knoten zusammensetzen. Das Aufstellen von indexnotierten Ausdrücken erfordert einen algebraischen Ausdruck als Assemblierungsblaupause. Dieser wird rekursiv durchlaufen und mittels der implementierten Algebrafunktionen in Indexnotation nachgebaut. Dabei werden die Dimensionen nicht eigenständig ermittelt sondern einfach aus dem algebraischen Ausdruck kopiert.

3.3.7 Templatesystem

Für das Arbeiten mit algebraischen Ausdrücken implementiert Baldur ein Templatesystem. Als Template wird hier ein algebraischer Ausdruck bezeichnet, der von externen, template-ierten Knoten abhängig ist. Externe, template-ierte Knoten sind externe Knoten, die als Templates gekennzeichnet sind und damit andere algebraische Ausdrücke repräsentieren. Über Templates können dynamisch allgemeingültige Handhabungsregeln für beliebige algebraische Tensorausdrücke implementiert werden. Baldurs Templatesystem verfügt dabei über mehrere Unterarten von template-ierten, externen Knoten. Für die Abgleiche überlädt Baldur intern den `==` Operator. Dieser wird über die hergestellte Alberich-Distribution dem Nutzer direkt freigeschaltet. Damit können Templatedarstellungen und deren Fähigkeit, algebraische Ausdrücke zu repräsentieren, dynamisch nachvollzogen werden. Die Abgleichsfunktion arbeitet dabei rekursiv. Dabei wird bei Gleichheit der Ausdrücke so lange simultan abgeglichen, bis eine Abweichung identifiziert wird oder der Ausdruck vollständig abgeglichen ist. Die Abgleichsfunktion vergleicht für assoziative Operationen wie der Addition und Skalarmultiplikation zusätzlich die Gleichheit für alle möglichen Permutationen der Abhängigkeiten. Tabelle 1 führt die verschiedenen template-ierten Knoten auf, die Baldur implementiert.

Tabelle 1: Verschiedene template-ierte Knotentypen des Templatesystems

Template-ierter Knoten	Beschreibung
`Standardtemplates`	Repräsentieren beliebiger Ausdrücke
`Argtemplates`	Repräsentieren beliebiger externer Knoten
`Instanztemplates`	Repräsentieren externer Knoteninstanzen
`Konstantentemplates`	Repräsentieren skalarer, ausgeschriebener Konstanten

Externe Standardtemplate-Knoten repräsentieren beliebige algebraische Tensorausdrücke einer konkreten Stufe. Diese wird bei der Konstruktion des Templateknotens angegeben. Wird die `-1` als Stufe übergeben, repräsentiert der Knoten beliebig stufige Ausdrücke. Argtemplate-Knoten repräsentieren beliebige externe Knoten einer konkreten Stufe. Auch diese Templateknoten erhalten die Stufe als Konstruktorinputargument und werden für die Übergabe der Stufe `-1` verallgemeinert. Instanztemplate-Knoten repräsentieren konkrete Instanzen externer Knoten. Bei

der Konstruktion wird das Label des externen Knotens übergeben, der repräsentiert werden soll. Der Instanztemplate-Knoten verkörpert damit alle externen Knoten unabhängig von Stufe und Dimensionen, die mit dem hinterlegten Label benannt sind. Konstantentemplate-Knoten repräsentieren externe Knoten, die skalare Konstanten darstellen. Damit ein externer Knoten als skalare Konstante gilt, muss er mit dem konkreten Wert der Konstante konstruiert werden. Werden Instanzen der beschriebenen, template-ierten Knoten in einen Ausdruck miteinbezogen, ist dieser ein Template. Damit beschreibt der Ausdruck keinen konkreten, algebraischen Term mehr, sondern eine Schar oder Menge an Ausdrücken.
Die Implementierung eines Regelsystems über Templates erfordert Register. Dabei genügen in den meisten Fällen Maps, die Templateausdrücke als Schlüssel nutzen. Die Registrierung einer Regel erfolgt über die Angabe eines Templateausdrucks und den Umsetzungsanweisungen der Regel. Soll das Regelsystem einen Ausdruck verarbeiten, wird von vorne nach hinten durch das Register iteriert und der aktuelle Schlüssel mit dem Ausdruck verglichen. Repräsentiert der Schlüssel den Ausdruck, wird die hinterlegte Regel angewendet. Da ein Templateregister unter Umständen mehrere Templates enthält, die einen gegebenen Ausdruck repräsentieren, implementiert Baldur eine konkrete Templateüberladungslogik, die über die Sortierung des Registers umgesetzt wird. Für die Sortierung der Register überlädt Baldur backendintern den zuständigen < Operator. Dieser wird innerhalb der Sprachdistribution zusätzlich freigeschaltet. Damit kann die Überladung dynamisch erprobt und nachvollzogen werden. Die Überladung der Register stellt dabei sicher, dass für einen Ausdruck immer das Template gefunden wird, das den Ausdruck am präzisesten repräsentiert. Deshalb geschieht die Überladung nach Spezifizierung. Templates, die andere Templates spezifizieren, werden weiter nach vorne sortiert als die Templates, die sie spezifizieren. So wird für Ausdrücke, die durch das spezifizierte Template verkörpert werden, die hinterlegte spezifizierte Regel gefunden. Für Ausdrücke, die nicht durch das spezifizierte Template repräsentiert werden, wird die Iteration durch das Register fortgesetzt und die Regel für das hinterlegte allgemeinere Template gefunden. Die Überladung berücksichtigt mehrere Faktoren, die einer konkreten Hierarchie folgen. Zudem arbeitet sie rekursiv. Das bedeutet, dass die Überladungslogik Ausdrücke bis in die Tiefe abgleicht, in der sie Unterschiede innerhalb der Spezifizierung findet.

Faktoren, die die Überladung bestimmen

- Template-ierung
- Templatevariante
- Knotentyp
- Stufenkonkretisierung
- Menge an einzigartigen externen Knoten

Der erste Faktor ist die Template-ierung des Ausdrucks, also ob es sich dabei um ein Template handelt oder nicht. Dabei überladen nicht template-ierte Ausdrücke, template-ierte Ausdrücke. Nicht template-ierte Ausdrücke sind Instanzen und nicht dazu fähig andere Ausdrücke zu repräsentieren. Damit sind sie in jedem Fall spezifischer als Templateausdrücke, die ganze Ausdrucksscharen verkörpern. Die Variante des externen, template-ierten Knotens ist für die Überladung ebenfalls ausschlaggebend. Das liegt daran, dass bereits die einzelnen Templateknotentypen einander spezifizieren und verallgemeinern. Der Standardtemplate-Knoten ist in der Lage beliebige Ausdrücke zu repräsentieren, darunter algebraische Operationen,

externe Knoten und Container. Der Argtemplate-Knoten repräsentiert hingegen nur externe Knoten. Damit spezifiziert er den Standardtemplate-Knoten und überlädt diesen. Der Instanztemplate-Knoten spezifiziert wiederrum den Argtemplate-Knoten, da er nur externe Knoten repräsentiert, die durch ein konkretes Label benannt sind. Der Konstantentemplate-Knoten überlädt den Argtemplate-Knoten ebenfalls. Da Konstantentemplate-Knoten und Instanztemplate-Knoten einander weder spezifizieren noch verallgemeinern, werden sie gegenseitig nicht explizit überladen. Die Überladung erfolgt zwar indirekt mit der Überladung der Tensorstufe (`0` und `-1`), diese ist aber redundant. Noch spezifischer sind externe Instanzknoten, die sowohl Instanztemplate-Knoten und Konstantentemplate-Knoten überladen. Ein weiterer Faktor, der die Überladung von Templateregistern bestimmt, ist der Knotentyp. Da Knoten algebraischer Operationen externe Knoten spezifizieren überladen sie diese. Die Tensorstufe wird ebenfalls bei der Überladung berücksichtigt. Das hat in erster Linie den Grund, dass Standard- und ArgTemplates eine Stufenkonkretisierung zulassen. Ist die `-1` als Tensorstufe in einem Standardtemplate-Knoten hinterlegt, repräsentiert dieser Ausdrücke beliebiger Stufen. Standardtemplate-Knoten mit einer Stufe größer `-1` verkörpern ausschließlich Ausdrücke der konkreten Tensorstufe. Um diese Spezifizierung abzubilden, überladen Ausdrücke mit einer geringeren Tensorstufen Ausdrücke mit einer höheren. Die Menge an einzigartigen externen Knoten ist ebenfalls ein Faktor, nachdem überladen wird. Sind zwei Ausdrücke strukturell identisch, wird nach der Menge an einzigartigen externen Knoten überladen. Dabei spezifiziert der Ausdruck mit weniger einzigartigen externen Knoten, den Ausdruck mit mehr einzigartigen externen Knoten.

Im Folgenden wird anhand mehrerer Beispiele gezeigt, wie sich die einzelnen Faktoren auf konkrete Überladungsbeispiele auswirken. Tabelle 2 zeigt die Überladung mehrerer Templatepaare.

Betrachtet werden folgende externe Knoten:

- Externe Standardtemplate-Knoten : `StdTmplA`, `StdTmplB`
- Stufenkonkretisierte Standardtemplate-Knoten : `StdTmplScalar`
- Externe Argtemplate-Knoten : `ArgTmplA`, `ArgTmplB`
- Externe Knoteninstanzen : `A`, `B`, `C`

Tabelle 2: Beispiele für Templateüberladungen nach verschiedenen Faktoren

Überladungsfaktor	Überladungsbeispiel
Template-ierung	`A + B < StdTmplA + StdTmplB`
Templatevariante	`ArgTmplA < StdTmplA`
Knotentyp	`StdTmplA + StdTmplB < StdTmplA`
Stufenkonkretisierung	`StdTmplScalar < StdTmplA`
Einzigartige externe Knoten	`StdTmplA + StdTmplA < StdTmplA + StdTmplB`

3.3.8 Ableitungen

Zur Ableitung von algebraischen Ausdrücken implementiert Baldur eine dynamisch konfigurierbare Ableitungslogik. Die Ableitungsmethode ist dabei eine Instanzfunktion des algebraischen Ausdrucks, die den Operanden, nach dem abgeleitet wird als Inputargument entgegennimmt

und die Instanz vor Ort modifiziert. Der Nutzer kann dabei dynamisch eigene Ableitungen und Ableitungsregeln implementieren. Die Implementierung erfolgt in zwei Laufzeitregister des Backends. Die Register enthalten dabei Ausdruckspaare als Schlüssel und für die Schlüssel hinterlegte Zielausdrücke. Das Ausdruckspaar enthält dabei die Operanden der Ableitungsoperation. Der Zielausdruck ist das Resultat der entsprechenden Ableitungen. Konkrete Umformungen oder Ersetzungen, die bei der Ableitung angestellt werden sollen und ausschließlich von konkreten, nicht template-ierten Instanzen abhängig sind, werden in das Ableitungsregister implementiert. Template-ierte Ableitungsvorschriften werden im Ableitungsregelregister hinterlegt. Bei Durchführung der Ableitung wird zuerst überprüft, ob der vorhandene Ableitungsfall durch eine registrierte Ableitung oder eine registrierte Ableitungsregel repräsentiert wird. Ist das der Fall, wird die Ableitung oder die Ableitungsregel auf den konkreten Fall angewendet. Im Falle der Übereinstimmung mit einer registrierten Ableitung kommt es bei der Ableitung zunächst zu der Ersetzung des aktuellen Ausdrucks durch den registrierten Zielausdruck. Wird die Übereinstimmung mit einer Ableitungsregel erkannt, müssen zusätzlich alle im Zielausdruck vorkommenden template-ierten externen Knoten durch die Größen substituiert werden, die sie repräsentieren. Dazu erstellt das Programm anhand der Ableitungsoperanden und der registrierten, template-ierten Entsprechungen im Register eine Substitutionsmap. Beim Erstellen dieser Map werden die Entsprechungen zusätzlich auf Konsistenz geprüft. So wird sichergestellt, dass alle Vorkommen jedes konkreten template-ierten Knotens innerhalb der registrierten, template-ierten Ableitungsoperanden die identische Instanz repräsentieren. Werden Inkonsistenzen bei der Erstellung der Substitutionsmap festgestellt, wird die Anwendung des Zielausdrucks terminiert und das nächste registrierte Ableitungstemplate wird darauf geprüft, ob es die angegebenen Operanden repräsentieren kann. Eignet sich keine registrierte Ableitung oder Ableitungsregel für die Darstellung der Operanden führt das Backend die Ableitung selbst durch. Dabei unterscheidet das Backend drei verschiedene Fälle, die es in dieser Reihenfolge überprüft. Wird ein beliebiger Ausdruck nach sich selbst abgleitet, wird der aktuelle Knoten durch die Identität der entsprechenden Dimension ersetzt. Bei Ableitung eines Operationsknotens nach einem anderen beliebigen Ausdruck, ersetzt die Funktion den aktuellen Knoten, durch die Anwendung der Ketten- oder Produktregel für den aktuellen Fall. Dabei ruft die Ableitungsfunktion rekursiv weitere Ableitungszuweisungen auf. Der letzte Fall ersetzt beliebige Ableitungen, die durch die bisher geprüften Fälle nicht behandelt werden konnten durch sogenannte rohe Ableitungen. Dieser Fallback umfasst dabei auch die Ableitungen von externen Knoten nach beliebigen Ausdrücken, die nicht durch registrierte Ableitungen oder Ableitungsregeln weiter spezifiziert worden sind. Als rohe Ableitung wird hier ein Knoten bezeichnet, der die beiden Operanden der Ableitung als Abhängigkeiten enthält und als Ableitung markiert ist. Algorithmus 21 zeigt diese Fallunterscheidung in der Ableitungslogik exemplarisch.

Algorithmus 21: Funktionsweise der tensoralgebraischen Ausdrucksableitung

```
1  void TensorExpression::diffAssign(const TensorExpression& other){
2
3      if(tensorExpressionDiffs.contains(std::make_pair(*this, other))){
4
5          *this = tensorExpressionDiffs[std::make_pair(*this, other)]; return;
6      }
7
8      static substitutionMap subsMap = {}; bool IsRepresentableByTemplate = false;
9      auto it = tensorExpressionDiffTemplates.begin();
10
11     // Check ob Expression durch ein abgespeichertes Template repräsentiert wird
12     for (; it != tensorExpressionDiffTemplates.end(); ++it) {
```

```
13
14          if (it->first.first == *this && it->first.second == other)
15          {
16              // Konsistenzabgleich der Darstellung f(subsMap, ...)
17              bool isRepresentationConsistent = ...
18              if(isRepresentationConsistent){ IsRepresentableByTemplate = true; break; }
19          }
20      }
21
22      if(IsRepresentableByTemplate){
23
24          TensorExpression res = it->second;      // Ergebnis wird durch Zielausdruck ersetzt
25          replaceBySubstitutions(res, subsMap);   // Anwenden der Substitutionen auf Ergebnis
26
27          *this = res.rebuild();  // Neubau des Resultats und Modifizierung der Instanz
28      }
29      // Bei Gleichheit wird die Identität als Ableitung genutzt
30      else if(*this == other){ *this = TensorExpression("Identity", 2 * this->tensorOrder); }
31      else if(Relation == TkType::Operator){ /* Kettenregel, Produktregel */ }
32      // Fallback mit roher Ableitung
33      else{ rawDiffAssign(other); }
34  }
```

Durch diese Fallunterscheidung und die Behandlung der einzelnen Fälle werden abgeleitete Ausdrücke so weit behandelt, bis nur noch rohe Ableitungsknoten übrig sind. Für die vollständige Weiterverarbeitung empfiehlt es sich, Ableitungen und Ableitungsregeln so zu registrieren, dass die rohen Ableitungsknoten eliminiert werden. Für den Export in dateiexterne Formate ist die Eliminierung der rohen Ableitungen teilweise notwendig. Die Eliminierung lässt sich über sehr allgemein formulierte Ableitungstemplates auch automatisieren, in dem unbekannte Ableitungen, die sonst rohe Ableitungsknoten erzeugen würden mit Nullmatrizen ersetzt werden.

3.3.9 Vereinfachungen

Die Vereinfachung von algebraischen Ausdrücken lässt sich ebenfalls dynamisch über ein templatebasiertes Regelsystem steuern. Das verwendete Templateregister hinterlegt für einen Templateausdruck als Schlüssel einen Zielausdruck. Bei der Vereinfachung wird der zu vereinfachende Ausdruck rekursiv durchlaufen und es wird für jeden Knoten überprüft, ob er durch eine der registrierten Templateinstanzen im Vereinfachungsregister dargestellt wird. Ist das der Fall, wird eine Substitutionsmap erstellt, die bei ihrer Zusammensetzung auf Konsistenz geprüft wird. Nach dem Zusammenbau der Substitutionsmap wird der aktuell betrachtete Ausdruck durch den registrierten Zielausdruck ersetzt und die Substitutionen aus der Substitutionsmap werden auf dem ersetzten Zielausdruck angewendet. Im Anschluss wird der Ausdruck neu aufgebaut, um zu garantieren, dass keine Templateartefakte, wie beispielsweise eine invalide Tensorstufe im Ausdruck verbleiben. Die Vereinfachungsfunktion wird dabei in Schleife so lange aufgerufen, bis im Ausdruck keine Übereinstimmungen mit registrierten Templates mehr gefunden werden.

3.3.10 Ergebnisexport

Baldur implementiert zunächst nur eine Exportmethode für indexnotierte Tensorausdrücke. Dabei wird ein Julia-Skript hergestellt, das den indexnotierten Ausdruck in Abhängigkeit der externen Knoten evaluiert. Die Exportierung als Julia-Skript ermöglicht dabei die Einbindung der ermittelten Ergebnisse in bereits existierende Projekte. Das generierte Julia-Skript besteht aus mehreren Hilfsfunktionen und der zentralen Evaluierungsfunktion. Die Evaluierungsfunktion

stellt zunächst sicher, dass die erwarteten Inputs den erwarteten Dimensionen entsprechen und konstruiert Instanzen von verwendeten Standardtensoren wie Identitäts- oder Nulltensoren. Im Anschluss wird der indexnotierte Ausdruck evaluiert. Für die Evaluierung werden die von der Bibliothek `Tensoroperations.jl` implementierten Makros genutzt. Die Bibliothek bietet dabei eine angenehme Notation und eine stark optimierte Evaluierung. Da die Makros jedoch nicht unbegrenzt lange Ausdrücke auswerten können, wird eine Logik implementiert, die Teile des Ausdrucks in Abhängigkeiten substituiert. Die rekursive Skriptgenerierung implementiert dazu einen booleschen Wert als Inputparameter, der mitteilt, ob die Substitution des aktuellen Knotens erzwungen werden soll. Ist das der Fall, vergibt sie der Abhängigkeit einen einzigartigen Namen, hängt die Deklaration an einen statischen Abhängigkeitenstring an, ersetzt den aktuellen Knoten durch einen externen Knoten mit dem erzeugten Label und ruft die Funktion erneut für den angepassten Knoten auf. Die Substitution wird dabei erzwungen, wenn der betrachtete Ausdruck eine bestimmte eingestellte Komplexität übersteigt. Da `TensorOperations.jl` teilweise Probleme mit dem Einpflegen von Skalaren in die Notationen der komplexeren Ausdrücke hat, werden skalare Operanden ebenfalls substituiert.

3.3.11 Language Support

Für die Einbindung der Sprache in bestehende digitale Ökosysteme implementiert Baldur einen Language Server, der nach dem offiziellen LSP (Language Server Protokoll) arbeitet. Zudem wird eine Erweiterung für den Codeeditor Visual Studio Code (VS Code, Microsoft Corporation) bereitgestellt, die den Interpreter und Language Server einbettet.

4 Anwendung

4.1 Alberich

4.1.1 Distributionsherstellung

In diesem Kapitel wird dargestellt, wie sich eine Alberich-Sprachdistribution herstellen lässt. Dabei wird allerdings keine detaillierte Schritt-für-Schrittanleitung ausgearbeitet, die alle Eventualitäten und Nischenfeatures abdeckt. Für sehr spezifische Fragen sollte deshalb die Alberich-Dokumentation oder der Baldur-Quellcode zurate gezogen werden. Da Baldur eine öffentlich verfügbare, voll funktionsfähige Alberich-Distribution ist, bietet der Quellcode dieses Projekts Aufschluss über konkrete, spezifische Realisierungen und Implementierungen und beantwortet damit die meisten Fragen die auftauchen können. Zum Schreiben einer Distribution muss der Alberich Quellcode zunächst von GitHub geklont und in das eigene C++-Projekt eingebunden werden. Konkrete Anleitungen und Hilfestellungen gibt es hierfür direkt im GitHub Repository.

4.1.1.1 Backendfreischaltungen

Nach der Einrichtung können die Backendfreischaltungen geschrieben werden. Dabei wird die Fehlerausgabe dieser Funktionalitäten idealerweise über das Anschließen der Funktion Fehlerkontextfunktion `getErrorContext` erweitert. Der Anschluss ist zwar nicht zwingend erforderlich, verbessert die Einordnung backendspezifischer Fehlerausgaben jedoch massiv. Weitere Informationen zu der Funktion und deren Anschluss sind in Kapitel 3.2.3.4.6 aufgeführt. Die Backendfreischaltung beginnt mit der Registrierung der Backendobjekte. Die Registrierung eines Backendobjekts erfordert die Deklarierung einer verpackenden Objektklasse, die von den in Kapitel 3.2.1.1 näher beschriebenen abstrakten Klassen erbt. Die genaue Einstellung der Behandlung dieser Objekte bei der Evaluierung geschieht anhand des Überschreibens der virtuellen Funktionen des `IObjekt`. Algorithmus 22 demonstriert die Deklarierung einer solchen Objektklasse für die Freischaltung des Typens `int`. Da das Interface `INativeObject` die meisten Deklarierungen übernimmt, müssen hier nur Konstruktoren für die Erstellung von selbstverwalteten und uniformisierten Integern implementiert werden, sowie einer Funktion, die die individuelle Ausgabe des Typens definiert. Zudem wird der Funktionsrumpf der Funktion `bool setUpClass` definiert. Die Deklarierung der Funktion findet an anderer Stelle statt und übernimmt die Typenregistrierung und Funktionsregistrierungen.

Algorithmus 22: Implementieren einer Typenklasse

```
1  namespace types{                                  // Verhindert Bennenungskonflikte
2
3    class INT : public INativeObject<INT, int>{     // Objekt Klasse für Verpacken des Integers
4
5      static bool setUpClass();                     // Typen und Funtkionsregistrierung
6
7      INT() = default;                              // Defaultkonstruktor
8      INT(int i) : INativeObject(i){}               // Konstruktor für selbstverwaltetes Objekt
9      INT(int* Ptr) : INativeObject(Ptr){}          // Konstruktor für Uniformisierung
10
11     // Ausgabe des freigeschalteten Objekts
12     void print() const override{ LOG << getMember() << "i"; }
13   };
14 };
```

Die Typenregistrierung geschieht über die Methode `INativeObject::init`. Diese erwartet ein Typenkeyword und einen Defaultkonstruktor. Nach einer erfolgreichen Registrierung des Typens können eigenständige, statische und Attributsfunktionen definiert werden, die den registrierten Typen als Member, Inputargument oder Rückgabe verwenden. Dazu implementiert Alberich mehrere Makros, die den Schreibaufwand und alle notwendigen Recasts übernehmen. Tabelle 3 gibt eine Übersicht über die wichtigsten Makros, die Teile der Funktionsregistrierung automatisieren.

Tabelle 3: Standard Makros für Funktionsregistrierung

Funktion	Beschreibung
`FREG_ARGS`	Funktionsinputs des Registerformats
`FREG_ARG_TYPES`	Funktionsinputtypen des Registerformats
`ASSERT_IS_NO_MEMBER_FUNCTION`	Stellt sicher, dass Funktion keine Attributsfunktion ist
`ASSERT_IS_MEMBER_FUNCTION`	Stellt sicher, dass Funktion eine Attributsfunktion ist
`ASSERT_HAS_N_INPUT_ARGS(numArgs)`	Stellt sicher, dass registrierte Funktion bestimmte Anzahl an Inputs aufweist
`PREPARE_RETURNS`	Passt Returnvektor an Rückgaben der Funktion an, überschreibend
`APPEND_RETURNS`	Passt Returnvektor an Rückgaben der Funktion an, hängt an
`CLEAR_RETURNS`	Setzt den Returnvektor zurück
`GET_RETURN(CastType, Positon)`	Anlegen einer Variablen mit recast des Returns
`GET_ARG(CastType, Positon)`	Anlegen einer Variablen mit recast der Eingabe
`GET_MEMBER(CastType)`	Anlegen einer Variablen mit recast der Instanz
`RECAST_INPUT_INPLACE(...)`	Recast eines Inputs vor Ort

Die Funktionsregistrierung wird dabei über die Funktionen `registerFunction`, `registerMemberFunction` und `registerStaticFunction` gepackt und erfordert die Übergabe des Funktionsnamens, der Inputtypenindizes, der Rückgabetypenindizes und der eigentlichen Funktion im entsprechenden Format. Soll eine statische oder Attributsfunktion registriert werden, ist die zusätzliche Angabe des Typenindizes der entsprechenden Instanz bzw. Klasse notwendig. Dabei muss die Funktion in das erwartete Funktionsregisterformat überführt werden. Das kann zum Beispiel durch das Packen des Aufrufs der Backendfunktion in eine Lambdafunktion passieren, die sich um den Cast der Inputs und das Zuführen der eigentlichen Instanzen hinter den Inputs kümmert. Innerhalb der Projekte Alberich und Baldur implementierte Funktionsregistrierungen stellen häufig zuerst sicher, dass es sich bei den aufgerufenen Funktionen tatsächlich um Attributsfunktionen oder keine Attributsfunktionen handelt und dass die korrekte Anzahl an Inputparametern übergeben wird. Da die Überladung jedoch zuverlässig funktioniert sind diese Assertions redundant und können auch weggelassen werden. Relevant ist allerdings das Vorbereiten des Rückgabevektors, der Cast von Inputs, Returns und Instanz und die Deklarierung der Rückgabe. Auf die eigentlichen Instanzen hinter der Objektfreischaltung kann über `INativeObject::getMember()` zugegriffen werden. Algorithmus 23 zeigt die Registrierung der Objektklasse, die den Integer freischaltet und die Funktionsregistrierung eines zugehörigen Pseudokonstruktors.

Algorithmus 23: Registrieren einer Typenklasse und Funktion

```
namespace types{

  bool INT::setUpClass(){

    // Registrierung über von INativeObject implementierte Funktion
    if(!init("int", [](){ return new INT(); })){ return false; }

    // Pseudo Standardkonstruktor
    registerFunction("int", { /* Typenindizes Inputs */ }, [](FREG_ARGS){

      PREPARE_RETURNS;          // Returns an Rückgaben anpassen
      GET_RETURN(INT, 0);       // Recast von inputs[0], Speichern in Zeiger ret0
      ret0->getMember() = 0;    // Zugriff auf verpackten int über getMember
    }, {INT::typeIndex});       // Typenindizes Returns

    return true;
  }
}
```

Die Schreibweise ist dabei etwas sperrig, ermöglicht aber eine sehr effiziente Ressourcenhandhabung. Über den direkten Zugriff auf die Evaluierungsergebnisse können sehr genaue Informationen über die vorliegenden Eingaben abgefragt und die Logik entsprechend sehr genau angepasst werden. Zudem ermöglicht der uneingeschränkte Zugriff Recasts der erhaltenen Inputs. Des Weiteren können Funktionen für beliebige Inputargumenttypen und beliebig lange Inputargumentlisten registriert werden.

4.1.1.2 Backendregistrierung

Um die vorgenommenen Registrierungen abzusetzen, muss die Funktion, in der die Registeraufrufe angestellt werden, aufgerufen werden. Eine andere Möglichkeit ist die Registrierung der Funktion im Backendregister. Über `emplaceBackend` können boolesche Funktionspointer im Backendregister hinterlegt werden. Backends, die direkt per Default freigeschaltet werden sollen, können nach Registrierung über `fetchBackend` freigeschaltet werden. Wird das Backend nicht unmittelbar vom Programm freigeschaltet, kann der Nutzer es später selbst freischalten. Dazu muss dann das Label, welches bei der Registrierung angegeben wird, angeführt werden. Algorithmus 24 zeigt die Registrierung und optionale, unmittelbare Freischaltung des exemplarisch deklarierten Integer Backends.

Algorithmus 24: Registrieren eines Backends

```
RETURNING_ASSERT(setUpBackendRegister(), "Backendregister konnte nicht aufgesetzt werden",);

emplaceBackend("int", &types::INT::setUpClass);
fetchBackend("int");
```

4.1.1.3 Skriptausführung

Alberich bietet über die Funktion `executeProgram` eine unkomplizierte Möglichkeit Skripte der Distribution auszuführen. Die Funktion übernimmt dabei alle notwendige Schritte inklusive der Aufrufe von Präprozessor, Lexer, Parser und Evaluierer, sowie der Verwaltung der entsprechenden Ressourcen. Nach der Evaluierung werden die vom Skript zurückgegebenen Variablen kopiert, um zu verhindern, dass sie noch interpreterkontextinterne Abhängigkeiten haben, der

Interpreterkontext wird freigeben und die Kopien der Skriptrückgaben werden zurückgegeben. Sollen die Skriptrückgaben anderweitig behandelt werden, empfiehlt sich die Implementierung einer eigenen Funktion oder die Pseudorückgabe über eine Interaktion mit uniformisierten Größen und Containern. Die Funktion `executeProgram` bietet zudem die Möglichkeit, auf einem von der Distribution eingerichteten Scope zu arbeiten. So kann vor Aufruf der Funktion ein Scope mit benötigten Metavariablen und uniformisierten Größen eingerichtet und dieser an die Funktion übergeben werden. Dabei können backendspezifische Metavariablen und Uniforms von nicht unmittelbar freigeschalteten Backends auch in der registrierten Freischaltungsfunktion in den Distributionsscope implementiert werden. Wird ein Distributionsscope übergeben, nutzt die Funktion diesen als übergeordneten Scope des eigenen Basisscopes. Damit bleibt die Verwaltung des eingerichteten Distributionsscopes jedoch vollständig der Distribution überlassen. Algorithmus 25 zeigt die exemplarische Einrichtung eines distributionsverwalteten Scopes und dessen Übergabe an die skriptausführende Funktion.

Algorithmus 25: Anlegen eines Distributionsscopes und Weitergabe an den Interpreter

```
1  Scope distroScope;
2
3  // Uniformisierung einer backendspezifigen Größe
4  bool backendSpecificFlag;
5  distroScope.constructAndReturnVariable("backendSpecificFlag") \
6      ->constructByObject(new types::BOOL(&backendSpecificFlag))
7
8  // Möglichkeiten zur Freischaltung von Metaparametern
9  // . Keywordregistrierung -> konstante Größe, weniger effiziente Handhabung
10 // . Metavariablenkonstruktion -> veränderbare Größe, effizientere Handhabung
11 registerKeyword("__distroVersion__", new types::STRING("0.0.x"));
12 distroScope.constructVariable("__distroVersion__", new types::STRING("0.0.x"));
13
14 // Ausführung eines Skriptes, Übergabe des Distributionsscopes
15 executeProgramm(scriptPath, &distroScope);
```

4.1.1.4 Interpreterprogramm

Um den ausführbaren Distributionsinterpreter herzustellen, muss ein Programm geschrieben werden, dass die beschriebenen Backendregistrierungen und unmittelbaren Freischaltungen durchführt und den in Kapitel 4.1.1.3 beschriebenen Funktionsaufruf absetzt. Dabei empfiehlt sich ein einfaches nicht interaktives Kommandozeilenprogramm. Das Programm sollte sich dabei über Kommandozeilenargumente steuern lassen. Über die Argumente bekommt das Programm notwendige Informationen, wie die Art der auszuführenden Aufgabe und die für die Ausführung relevanten Steueroptionen übergeben. Für die Skriptausführung ist das unter anderem der Skriptpfad. Weitere mögliche Funktionalitäten des Interpreterprogramms sind die Ausgabe der Lizenz, der Version oder das Öffnen einer Dokumentation.

4.1.1.5 Einbindung in bestehende digitale Ökosysteme

Die Einbindung des Distributionsinterpreters in bestehende digitale Ökosysteme und Umgebungen ist häufig sehr ökosystemspezifisch. Allerdings gibt es für die Einbindung relevante Komponenten, die sich häufig ähneln.

Für Einbindung benötigte Standardkomponenten

- Interpreterprogramm
- Language Server
- Konfigurationsdateien

Die Implementierung der Konfigurationsdateien dient dabei dem statischen Syntaxhighlighting und der Mitteilung, mit welchen Argumenten der Interpreter angesprochen werden muss. Der Language Server übernimmt das dynamische Syntaxhighlighting, Autovervollständigungsvorschläge, Tooltips und mehr. Dabei sollte er das offizielle Language Server Protokoll (LSP) unterstützen. Die Implementierung des Language Supports einer Alberich-Distribution wird im Baldur Quellcode für den Editor Visual Studio Code durchgeführt.

4.1.2 Sprachreferenz

Dieses Kapitel gibt einen Überblick darüber wie Syntax und Semantik von Alberich Distributionen gestaltet sind und wie sie effizient geschrieben werden können. Alberich bietet dabei in mehreren Fällen zwei verschiedene Syntaxschreibweisen an. Dabei handelt es sich um explizite und implizite Schreibweisen. Die implizite Schreibweise ist dabei der in herkömmlichen Interpretersprachen verwendeten Syntax deutlich ähnlicher als die explizite. Sie gibt dem Interpreter Spielraum und überlässt ihm in einigen Fällen die Auswahl darüber, welche mikroskopischen Operationen durchgeführt werden. Die explizite Schreibweise hingegen ermöglicht tiefgreifendere Vorgaben innerhalb dieser Fälle und die eigenständige Auswahl der entsprechenden Operationen. Mit der expliziten Schreibweise geht eine bessere Lesbarkeit und eine genauere Steuerung des Programms einher. Allerdings kann eine suboptimale Nutzung der expliziten Schreibweise die Programmausführung auch ineffizienter machen und deutlich verlangsamen.

4.1.2.1 Skriptformatierung

Die Formatierung von Distributionsskripten wird nicht über Absätze oder Einrückungen vorgenommen, sondern über hierarchiebestimmende Elemente wie Klammern und Trennzeichen. Logische Anweisungsabschnitte müssen dabei durch das Semikolon voneinander separiert werden. Dabei muss allerdings nicht jedes Semikolon manuell gesetzt werden. Vor der lexikalischen, syntaktischen und semantischen Analyse wird das Skript von dem Präprozessor vorformatiert. Dieser schreibt automatisch hinter jede Zeile ein Semikolon. Dadurch ist das Abtrennen der Zeilen durch Semikolons von Hand zwar möglich aber nicht nötig. Nur wenn beispielsweise zwei logische Anweisungsabschnitte innerhalb einer Zeile angeführt werden, müssen diese durch ein Semikolon voneinander getrennt werden. Das automatische Setzen der Semikolons durch den Präprozessor kann für einzelne Zeilen unterdrückt werden. Das geschieht durch die Angabe der Präprozessordirektive \ am Zeilenende. Diese Direktive gibt an, dass sich die aktuelle Zeile in der nächsten fortsetzt und kein Semikolon zur Abtrennung der Zeilen voneinander eingefügt werden soll. So können zusammenhängende Abschnitte geschrieben werden, die sich über mehrere Zeilen erstrecken. Im Rahmen der folgenden Kapitel werden mehrere Syntax- und Semantikformate vorgestellt. Um zu gewährleisten, dass der Interpreter die einzelnen Parameter der Formate korrekt auseinanderhalten kann, ist es teilweise notwendig, diese durch Sektionen einzugrenzen. Sektionen können dabei durch die Eingrenzung von Codeabschnitten durch runde, eckige oder geschweifte Klammern hergestellt werden. Dabei muss nur darauf geachtet werden, dass innerhalb von Sektionen, die durch runde Klammern eingegrenzt werden eine andere Operatorenhierarchie

gilt. Innerhalb von Sektionen, die durch runde Klammern eingegrenzt werden, wird der Kommaoperator vor allen anderen Operatoren präferiert. Durch die Übergabe eines Codeabschnitts innerhalb runder Klammern wird die makroskopische Unterteilung des Abschnitts nach dem Kommaoperator forciert. Algorithmus 26 demonstriert die benötigte Skriptformatierung.

Algorithmus 26: Übersicht über die erwartete Skriptformatierung

```
1 ...         // Semikolon wird hier am Zeilenende automatisch gesetzt
2 ...; ...    // Unterteilung innerhalb der Zeile muss manuell vergenommen werden
3 ... + 2 \   // '\' Token unterdrückt automatische Formatierung.
4     + 3 \   // Gewährleistet zeilenübergreifenden, ununterbrochenen Ausdruck
5     + 4     // Semikolon wird dann in der letzten Zeile des Ausdrucks wieder automatisch gesetzt
```

4.1.2.2 Standardobjekte

Alberich selbst implementiert bereits Freischaltungen für mehrere Standardtypen. Diese Freischaltungen können bei Bedarf gelöscht und selbst implementiert oder erweitert werden. Auch eine Implementierung parallel zur Standardimplementierung ist möglich.

Freigeschaltete Standardobjekte

- Void
- Bool
- Integer
- Double
- String
- Args

Die Typen Bool, Integer, Double und String sind dabei Freischaltungen der Basistypen, die die Sprache C++ verwendet. Der Typ Void ist ein leeres Objekt. Void Variablen können dabei Objekte jedes Typens zugewiesen bekommen und werden damit automatisch recastet. Args Objekte beinhalten Vektoren von Evaluierungsergebnissen. Damit sind sie keinesfalls ein Ersatz für Vektoren oder Listen, die tatsächliche Variablen enthalten. Das liegt an ihrer Unfähigkeit Variablen richtlinienkonform zu besitzen. Sie eignen sich in erster Linie zum Packen und Formatieren anderer Strukturen, ohne dass dafür neue Variablen erstellt werden müssen. Die implementierten Standardtypen sind dabei allesamt uniformisierbar. Die Uniformisierung kann dabei auch für Attribute von freigeschalteten Objekten dieser Datentypen erfolgen.

4.1.2.3 Objekt Konstruktion

Die Objektkonstruktion bietet die Möglichkeit einer expliziten und einer impliziten Schreibweise. Die explizite Konstruktion von Instanzvariablen eines Objekttypens erfolgt über die Angabe des Keywords, unter dem der Typ registriert ist, und des Variablennamens. Dabei wird sichergestellt, dass die Variable in den aktuellen Scope geschrieben wird. Ist in diesem bereits eine Variable unter diesem Namen hinterlegt, kommt es zu einer Fehlerausgabe. Die implizite Konstruktion von Instanzvariablen erfolgt entweder über die Angabe des `void` Keywords vor dem Variablennamen oder ganz ohne Keyword. Die Angabe des `void` Keywords stellt dabei die Konstruktion

in den Scope sicher. Wird das Keyword weggelassen, kommt es nur zu einer Konstruktion, wenn noch keine Variable unter diesem Namen im aktuellen Scope und den entsprechenden Elternscopes gefunden werden kann. Die implizite Konstruktion ohne Typenkeyword führt dabei zur Konstruktion einer Void-Variablen. Zudem können Variablen über die Angabe mehrerer Variablennamen in eckigen Klammern hinter dem Typenkeyword sowohl implizit als auch explizit simultan konstruiert werden. Da Alberich über keine klassische Konstruktorensyntax verfügt, muss der Wert der konstruierten Variablen über eine Zuweisung gesetzt werden. Für die Standardtypen implementiert Alberich dazu mehrere Pseudokonstruktoren. Dabei handelt es sich um Funktionen, die Objekte des gleichnamigen Typen zurückgeben. Ausgeschriebene Zahlen, Strings und registrierte Keywords geben ebenfalls Objekte zurück, die den konstruierten Variablen zugewiesen werden können. Bei ausgeschriebenen Zahlen entscheidet die Angabe des Dezimalpunkts darüber, ob bei der Evaluierung ein Double oder Integer zurückgegeben wird. Zudem kann eine Zuweisung nur erfolgen, wenn der zugewiesene Wert und die Variable, die den Wert zugewiesen bekommen soll, vom gleichen Typen sind. Nur einer Void Variablen können beliebige Werte anderer Typen zugewiesen werden. Die Zuweisung führt dabei zu einem Recast der Void Variablen zum zugewiesenen Typen. Aus diesem Grund werden bei der impliziten Konstruktion Void-Variablen erstellt. Die Konstruktion von Referenzvariablen ist nach dem gleichen Schema, wie die Konstruktion der Instanzvariablen möglich. Dabei muss nur das Referenz-Keyword `ref` zwischen Typenkeyword und Variablennamen angegeben werden. Es sind dabei sowohl explizite und implizite, als auch Simultankonstruktionen möglich. Algorithmus 27 zeigt mehrere beispielhafte Konstruktoraufrufe.

Algorithmus 27: Explizite und implizite Konstruktion von Instanz- und Referenzvariablen

```
1 bool bVariable = true; int iVariable = 4            // Explizite Konstruktion Instanzvariablen
2 void strVariable = "..."; argsVariable = args()     // Implitzite Konstruktion Instanzvariablen
3 bool ref bRef                                       // Explizite Konstruktion Referenzvariablen
4 void ref iRef; ref strRef                           // Implizite Konstruktion Referenzvariablen
5 int [a, b, c]; void ref [a, b, c]                   // Simultane Konstruktion
```

4.1.2.4 Simultanverknüpfungen

Das parallele Arbeiten mit mehreren Evaluierungsergebnissen pro Rückgabe ermöglicht die simultane Verknüpfung mehrerer Operandenketten oder -paare über Operatoren. Da die meisten Operatoren per Default eine höhere Präferenz, als der Kommaoperator haben, kann dieser zur Abgrenzung der simultanen Operanden eingesetzt werden. Da die Anführung von runden Klammern die Präferenz des Kommaoperators jedoch forciert, müssen die Bestandteile der Simultanoperanden innerhalb von runden Klammern über weitere Elemente wie eckige Klammern eingegrenzt werden. Algorithmus 28 zeigt die Anwendung der Simultanverknüpfung.

Algorithmus 28: Übersicht über die Nutzungsmöglichkeiten von Simultanverknüpfungen

```
1 // Simultanverknüpfung -> a = 6, b = 9
2 int [a, b] = 0, 1 + 2, 3 + 4, 5
3
4 // Aufgrund der runden Klammern keine Simultanverknüpfung -> a = 0, b = 3, c = 7, d = 5
5 int [a, b, c, d] = (0, 1 + 2, 3 + 4, 5)
6
7 // Erzwungene Simultanverknüpfung in runden Klammern, beide Versionen funktionieren
8 // -> a = 6, b = 9
9 int [a, b] = ([0, 1 + 2, 3 + 4, 5])           // erste Schreibweise
10 int [a, b] = ([0, 1] + [2, 3] + [4, 5])       // zweite Schreibweise
```

4.1.2.5 Berücksichtigung der Operatorenhierarchie

Beim Schreiben von Alberich-Distributionsskripten muss zudem die übergebene Operatorenhierarchie berücksichtigt werden. Da es keine Operatoren auf dem gleichen Hierarchielevel geben kann, empfiehlt sich eine sehr explizite Klammersetzung. Ein Beispiel hierfür ist die Nutzung des `+` und `-` Operators. In vielen anderen Sprachen haben diese Operatoren die gleiche Präferenz. Deshalb lassen sich dort Operandenketten über Aneinanderreihung beider Operatoren verketten, ohne das eine explizite Klammersetzung erforderlich ist. Der Ausdruck wird dabei strikt von links nach rechts in Operanden aufgeteilt, unabhängig davon, welcher Operator jeweils angewendet worden ist. So kann der Ausdruck `a + b - c + d` eindeutig als `((a + b) - c) + d` interpretiert werden. In Alberich hingegen besitzt jeder Operator eine eindeutige Präferenz, die über seine Position in der Operatorenhierarchie festgelegt ist. Kommen in einem Ausdruck verschiedene Operatoren vor, wird er zuerst nach Operanden des Operators mit der höheren Präferenz aufgeteilt und die Operanden dieser makroskopischen Operation dann in Operandenketten des Operators der niedrigeren Präferenz. Die Auswertung des Ausdrucks `a + b - c + d` ist dabei je nach Operatoren-Präferenz entweder `(a + b) - (c + d)` oder `a + (b - c) + d`. Deshalb ist für die links-nach-rechts-äquivalente Auswertung hier die explizite Klammersetzung mit `((a + b) - c) + d` oder dem Einpflegen des negierten `c` als Additionsoperand mit `a + b + (-c) + d` erforderlich.

4.1.2.6 If-Syntax

Aufgrund des parallelen Arbeitens mit mehreren Evaluierungsergebnissen und der simultanen Verknüpfung über Operatoren, braucht die Sprache eine If-Syntax, die mit Eingaben dieses Formats umgehen kann. Dazu werden vier verschiedenen if Konventionen verwendet. Für alle Fälle, die sich mit diesen Konventionen nicht abdecken lassen, muss eine eigene Logik implementiert werden. Das funktioniert über die Konstruktion einer Args-Variable. Diese kann beliebig viele Evaluierungsergebnisse gleichzeitig verwalten.

Implementierte If-Konventionen

- if
- nIf
- rIf
- xIf

Die einzelnen If-Konventionen sind dabei registrierte Funktionen, die alle innerhalb und außerhalb der klassischen If-Syntax verwendet werden können. Das hier verwendete `if` entspricht dabei der Standardkonvention, es akzeptiert einen Input und führt die nachgestellte Sektion aus, wenn der evaluierte Input `true` ist. Das `nIf` hingegen kann mehrere Inputs entgegennehmen. Es verknüpft diese über das logische Und. Auch `rIf` und `xIf` akzeptieren mehrere Inputs, `rIf` verknüpft diese dabei über das logische Oder und `xIf` über das logische, exkludierende Oder. Die If-Syntax erfordert dabei eine klare Trennung der einzelnen Abschnitte. Dazu können Absätze oder Semikolons verwendet werden. Algorithmus 29 demonstriert die Verwendung der If-Syntax von Alberich.

Algorithmus 29: Übersicht über die If-Syntax

```
1 if(false){ ... }
2 else nIf([a, b, c == 1, 2, 3]){ ... } // wird ausgeführt, wenn alle Parameter gleich sind
3 else rIf([a, b, c == 1, 2, 3]){ ... } // wird ausgeführt, wenn ein Parameter oder mehr gleich sind
4 else xIf([a, b, c == 1, 2, 3]){ ... } // wird ausgeführt, wenn genau ein Parameter gleich ist
5 else if(false){ ... }; else{ ... }    // Abtrennung durch Semikolon
```

4.1.2.7 Speicherverwaltungssemantik

Alberich ermöglicht über eine explizite Speicherverwaltungssemantik die Möglichkeit die Variablenhandhabung mikroskopisch zu steuern. Dabei nutzt diese explizite Schreibweise vier zentrale Speicherverwaltungsoperationen.

Zentrale Elemente der expliziten Speicherverwaltung

- Kopieren, Operator `<+`
- Verschieben, Operator `<-`
- Vertauschen, Operator `<>`
- Referenzieren, Operator `<<`

Die Operationen werden dabei in erster Linie für binäre Verknüpfungen genutzt. Der linke Operand ist dabei immer der Empfänger der Operation und der rechte die Quelle. Grundsätzlich sind die Operationen, abgesehen von dem Vertauschungsoperator, auch innerhalb von unären Verknüpfungen nutzbar. Dabei ist der angeführte Operand immer die Quelle. Der Empfänger der unären Verwaltungsoperation ist der RValue des erzeugten Evaluierungsergebnisses. Die Operationen Kopieren, Verschieben und Vertauschen beziehen sich dabei immer auf die besessenen oder referenzierten Instanzen. Das Referenzieren, also das Hinterlegen des Zeigers auf eine Instanz in einer Referenzvariablen, geschieht über die Operation Referenzieren. Dabei muss der Empfänger der Operation eine Referenzvariable sein. Bei der Operation wird die besessene oder referenzierte Instanz der Quelle referenziert. Die implizite Schreibweise nutzt den Zuweisungsoperator `=`. Dieser packt die Operationen Kopieren und Verschieben. Erhält der Zuweisungsoperator einen RValue als Quelle wird dieser verschoben. Ist die Quelle ein LValue, wird der Wert kopiert. Ein weiterer Operator, der für Zuweisungen genutzt werden kann, ist der Walrus-Operator. Alberich implementiert diesen über die Sequenz `=>`. Die Überladungsfunktion des Walrus-Operators führt eine Standardzuweisung durch. Zusätzlich werden die LValues der konstruierten Variablen zurückgegeben. Das sorgt dafür, dass die Verwendung dieses Operators in einigen Fällen deutlich kompaktere Schreibweisen ermöglicht. Algorithmus 30 zeigt mehrere Beispiele für die Verwendung expliziter und impliziter Speicherverwaltungssemantik.

Algorithmus 30: Übersicht über explizite und implizite Speicherverwaltungssemantik

```
1 // Explizite Verwaltung
2 int [a, b, c] <+ d, e, f        // Kopieren
3 int [a, b, c] <- d, e, f        // Verschieben
4 int [a, b, c] <> d, e, f        // Vertauschen
5 ref [g, h, i] << d, e, f        // Referenzieren
6 ...      [<+a, <-b, <<d]        // unäre Verwendung
7
8 // Implizite Verwaltung
9 int [a, b] = 1, d               // Interpreter entscheidet eigenständig [Verschieben, Kopieren]
10
11 // Kompakte Schreibweise durch Verwendung des Walrus-Operators
12 ref [c, d] << { int [a, b] => 1, 2 }
```

Die Nutzung expliziter Speicherverwaltungssemantik bedeutet eine gewisse Verantwortung. Dazu gehört die rechtzeitige Freigabe von Ressourcen und das Beseitigen ungültiger Ressourcen. Wird ein LValue als Quelle für eine Verschiebung genutzt, wird er bei diesem Vorgang invalidiert und der ursprüngliche Wert des Empfängers wird gelöscht. Referenzen, die die Quelle referenzieren, werden übernommen und zeigen nach der Verschiebung auf den Empfänger. Referenzen, die vor der Verschiebung auf den Empfänger gezeigt haben, werden durch die Verschiebung invalide. Aus diesem Grund ist es nötig die ursprünglichen Empfängerreferenzen über `dref` zu dereferenzieren. Die invalidierte Quellvariable sollte entweder nicht mehr genutzt oder besser ganz gelöscht werden. Da jede Beteiligung der Variablen in einer Operation oder einem Funktionsaufruf zu einem Abbruch führt, kann die Variable nur noch über Angabe ihres Namens aus dem Scope gelöscht werden. Die Verschiebung von LValues eignet sich in erster Linie, wenn die Quelle nicht mehr genutzt oder durch das Verlassen des lokalen Scopes im Anschluss gelöscht wird. Eine alternative Operation, die die Quellvariable nicht invalidiert, ihren Wert aber trotzdem verschiebt, ist die Vertauschungsoperation. Algorithmus 31 zeigt die erforderliche Ressourcenfreigabe nach der Verschiebung eines LValues.

Algorithmus 31: Korrekte Handhabung durch LValue Verschiebung invaliderter Ressourcen

```
1 int [a, b] <- 1, 2    // Verschiebung der beiden Werte in die konstruierten Variablen
2 ref [c, d] << a, b    // Referenzierung
3
4 dref c                // Dereferenzierung, referenzierte Instanz wird durch Verschiebung invalidert
5 a <- b                // Verschiebung
6
7 delete "b"            // Wenn b nicht mehr angeführt wird muss die Variable nicht gelöscht werden,
8                       // wenn die Variable weitergenutzt werden soll, ist das Löschen notwendig
9
10 int b = ...          // Neukonstruktion
```

4.1.2.8 Funktionsdeklarierung

Die Funktionsdeklarierung geschieht über das Keyword `decl`, der Angabe eines Labels, einer Inputparametersektion und einer auszuführenden Sektion. Dabei werden die Inputparameter der Parametersektion entsprechend der Standardkonstruktionssyntax aus Kapitel 4.1.2.3 angelegt. Wird ein Inputparameter dabei als Instanzvariable konstruiert, wird die entsprechende Eingabe beim Funktionsaufruf kopiert. Inputparameter, die als Referenzvariablen konstruiert werden, referenzieren übergebene LValues und kopieren übergebene RValues. Über die von Alberich implementierten Funktionen `isReference` und `isInstance` kann das Format, in dem eine

Funktionseingabe vorliegt, jederzeit überprüft werden. Die übergebenen Eingaben sind dabei aus dem Funktionskontext heraus vollständig bearbeitbar. Je nachdem, ob sie Referenz- oder Instanzvariablen sind, wird die Änderung in der referenzierten Variablen im Rückgabescope oder in der temporären Kopie, die im Funktionsscope liegt, beaufschlagt. Die Rückgabe wird über das Keyword `return` angestellt. Umfasst der Ausdruck hinter dem `return` mehr als eine Abhängigkeit, muss dieser in Klammern angegeben werden. Die Rückgabe eignet sich dabei für die unäre, explizite Speicherverwaltungssyntax. Ein Funktionsaufruf geschieht über den Aufruf des Funktionsnamens mit der Übergabe der Eingabeparameter innerhalb einer Sektion mit runden Klammern. Wird für die vorliegenden Inputparameter keine Überladung gefunden überprüft der Interpreter, ob er eine Überladung für beliebige Inputparameter finden kann, also für Void-Variablen oder für eine Args Variable. So können Funktionsdefinitionen bis zu einem gewissen Grad verallgemeinert werden. Zudem bietet Alberich eine lambdafunktions-ähnliche Syntax, die jedoch keine Inputparameter annimmt. Um eine ähnliche Funktionalität herzustellen, können Referenzen anstelle von Inputparametern verwendet werden. Algorithmus 32 zeigt die Deklarierung einer Funktion und die Nutzung der Pseudolambdasyntax.

Algorithmus 32: Funktionsdeklaration und Pseudo-Lambda Syntax

```
1 // Funktionsdeklarierung
2 decl fibonacci(int ref member){
3
4   // Pseudo Lambda Syntax für fallspezifische unäre Verschiebung bei Rückgabe
5   if(member <= 1){ return [if(isInstance(member)){ return [<-member] }; else { return member }] }
6
7   // hier wäre unäre Verschiebung redundant wegen RValue
8   return [fibonacci(member - 1) + fibonacci(member - 2)]
9 }
10
11 // Pseudo Lambda Syntax mit Pseudo Inputargumenten, die durch Referenzen gestellt werden
12 bool isMetric = ...
13 double ref temperature << ...
14
15 double temperatureKelvin = \
16   [if(isMetric){return [fromCelsius(temperature)]}; else {return [fromFahrenheit(temperature)]}]
```

4.1.2.9 Ausgaben

Die Ausgaben der Sprache erfolgen über die Funktionen `log` und `slog`. Beide Funktionen akzeptieren beliebig viele Inputargumente, die nacheinander ausgegeben werden. Die Spaced-Logging-Funktion oder `slog` schiebt dabei zwischen jedes Argument ein Leerzeichen. Zudem können genauere Informationen zu Evaluierungsergebnissen über die Funktion `logRes` ausgegeben werden. Algorithmus 33 demonstriert die Nutzung der erwähnten Ausgabefunktionen.

Algorithmus 33: Übersicht über die Ausgabefunktionen

```
1 int i = 5
2 log("Fibonacci[", i, "] : ", fibonacci(i))    // Ausgabe >> 'Fibonacci[5i] : 5i'
3 slog("Fibonacci[", i, "] :", fibonacci(i))    // Ausgabe >> 'Fibonacci[ 5i ] : 5i'
4 logRes(fibonacci(i))                          // Ausgabe >> 'Rvalue Variable : [tID 5    ] 5i'
```

4.1.2.10 Guards

Um Skripte wiederverwendbar zu gestalten und dafür zu sorgen, dass sie sowohl direkt ausgeführt, als auch aus anderen Skripten heraus eingebunden werden können, ermöglicht Alberich

die Nutzung von Include- und Main-Guards, also die Möglichkeit, aus dem ausgeführten Skript heraus abzufragen, ob es direkt ausgeführt oder von einem anderen Skript aus eingebunden wird. Das geschieht über die Methode `calledAs` und die Angabe eines Parameters, der einen Aufrufzustand repräsentiert. Für Main- und Include-Guards sind das `__MainProc__` und `__Include__`. Algorithmus 34 zeigt beispielhafte Verwendungen des Main und Include Guards.

Algorithmus 34: Verwendung von Main- und Include-Guards

```
1 // Wird ausgeführt wenn Programm direkt ausgeführt wird
2 if(calledAs(__MainProc__)){ ... }
3
4 // Wird ausgeführt wenn Programm aus einem anderen eingebunden wird
5 if(calledAs(__Include__)){ ... }
```

4.1.2.11 Schleifen

Alberich implementiert For- und While-Schleifen. Die Syntax orientiert sich dabei stark an der anderer Sprachen. Algorithmus 35 demonstriert die beiden Schleifenformate.

Algorithmus 35: Übersicht über verwendete Schleifenformate

```
1 for(initials, condition, increment){ ... }
2 while(condition){ ... }
```

Die einzelnen Argumente der Parametersektionen können dabei selbst ganze Sektionen oder Verarbeitungsalgorithmen sein. Schleifen können durch ein Breakstatement abgebrochen werden und durch ein Continuestatement in die nächste Schleifenausführung springen. Innerhalb des Schleifenscopes kann zu dem eine statische Sektion angeführt werden, die nur im ersten Schleifenframe aufgerufen wird. Da Schleifenscopes nach Ablauf der Schleife zerstört werden, muss darauf geachtet werden, externe Referenzen lokaler Variablen zu dereferenzieren. Algorithmus 36 zeigt ein Anwendungsbeispiel einer For-Schleife.

Algorithmus 36: Beispielhafte Verwendung der For-Schleife

```
1 // schleifenexterne Referenz
2 int ref expRef
3
4 for([int [i, j, k]; int [imax, jmax, kmax] = 5, 5, 5], \
5   !nIf([i, j, k] >= [imax, jmax, kmax]), \
6   [i++; if(i > imax){i = 0; j++}; if(j > jmax){i = 0; j = 0; k++}]){
7
8   // Wird beim ersten Durchlauf ausgeführt
9   static{ expRef << i }
10
11   // Dereferenzierung bei Schleifenabbruch
12   nIf([i, j, k] >= [[imax - 1], jmax, kmax]){ dref expRef }
13 }
```

4.1.2.12 Objektorientierte Programmierung

Alberich implementiert eine minimalistische objektorientierte Programmierung (OOP) ohne notwendige, kryptische Funktionsdefinitionen für Konstruktoren und Operatorüberladungen. Die Attribute werden dabei innerhalb von eigenständigen, unabhängigen Scopes gespeichert und alle Attribute sind öffentlich. Innerhalb der Objektdeklarierung können statische Sektionen genutzt werden, um statische Attribute und Funktionen zu deklarieren. Vererbung gibt es bislang

nicht. Bei der Deklaration von Attributsfunktionen kann dabei immer über das Keyword `this` auf die Instanz zugegriffen werden. Zugriffe auf Attribute und Attributsfunktionen erfolgen über den Operator `->`, Zugriffe auf statische Attribute und Funktionen über den Operator `>>`. Pseudokonstruktoren werden als externe Funktionen definiert, die das entsprechende Objekt erstellen, einrichten und in die Rückgabe verschieben. Die Verschiebung ist dabei unabhängig von der Größe des Objekts effizient, da bei der Verschiebung nicht das gesamte Objekt, sondern nur der Zeiger auf das Objekt verschoben wird, der auf 64-bit Systemen genau 8 Byte groß ist. Operatorüberladungen werden ebenfalls als externe Funktionen definiert. Die Überladungen sind dabei über die Überladungskonfiguration anstellbar und einsehbar. Die statische Sektion einer Klassendefinition kann als Pseudonamespace verwendet werden, da die statischen Scopes nach Evaluierung nicht gelöscht werden. Algorithmus 37 zeigt eine beispielhafte Nutzung von Alberichs objektorientierter Programmierung.

Algorithmus 37: Beispiel für Alberichs Objektorientierte Programmierung

```
1 struct Vector3{
2
3   // statische Attribute und Funktionen
4   static{ int constructedEntities = 0 }
5
6   // Attribute und Attributsfunktionen
7   double[x,y,z]
8   decl mul(double ref db){ x,y,z *= db,db,db }
9 }
10
11 // Konstruktor
12 decl Vector3(double ref [xIn, yIn, zIn]){
13
14   Vector3>>constructedEntities++
15   Vector3 vec; vec->[x,y,z] = xIn, yIn, zIn
16   return [<-vec]
17 }
18
19 // binäre Überladung für += und -=, verkettende Überladung für + und -
20 decl __addAssign__(Vector3 ref [a, b]){ a->[x,y,z] += b->[x,y,z] }
21 decl __subAssign__(Vector3 ref [a, b]){ a->[x,y,z] -= b->[x,y,z] }
```

4.1.2.13 Assertions

Alberich implementiert die Assertionsfunktion `assert`. Allerdings ist das Absetzen von Assertions auch über eine dedizierte Syntax möglich, die das Keyword `requires` verwendet. Die Verwendung dieser Syntax führt dazu, dass die Funktion im Hintergrund aufgerufen wird. Zudem bietet sie die direkte Möglichkeit, ohne manuelle Funktionsaufrufe sicherzustellen, dass eine erwartete, uniformisierte Größe tatsächlich aus dem Skript verfügbar ist. Algorithmus 38 demonstriert die erwarteten Assertionssyntaxformate.

Algorithmus 38: Übersicht über Assertionsformate

```
1 // Formate
2 assert(condition, errorMessage)         // direkter Aufruf der assert Funktion
3 requires condition errorMessage         // indirekter Aufruf der assert Funktion
4 requires typeKeyword uniform unifLabel  // Spezifische Syntax zum Sicherstellen eines Uniforms
5
6 // Beispiele
7 assert(isMetric, "Temperaturen müssen im metrischen Format übergeben werden")
8 requires {divisor != 0} "Division erfordert einen Divisor ungleich Null"
9 requieres bool uniform backendSpecificFlag
```

4.1.2.14 Einbinden von Inhalten

Das Einbinden von skriptexternen Inhalten erfolgt über das Keyword `fetch`. Beim Einbinden muss zusätzlich die Art der Ressource angegeben werden, die eingebunden werden soll. Zuletzt wird der Pfad oder Schlüssel hinterlegt, unter dem die Ressource gefunden werden kann. Dabei ist sowohl die Angabe einer einzelnen Ressource, als auch die von mehreren möglich. Aktuell ist das Einbinden von registrierten Backends und von anderen Skripten der Sprache möglich. Das Backend `core`, dass von Alberich selbst freigeschaltet wird und Standardobjekte und -funktionen zur Verfügung stellt, wird automatisch eingebunden. Die Einbindung von Skripten erfordert dabei die Angabe der relativen Pfade zu den entsprechenden Skripten. Algorithmus 39 zeigt das Einbinden mehrerer Backends und externer Skripte.

Algorithmus 39: Einbinden externer Ressourcen in Distributionsskripte

```
1 fetch backend ["expBackend1", "expBackend2"]
2 fetch script ["expScript1.alb", "expScript2.alb"]
```

4.1.2.15 Metavariablen

Neben uniformisierten Größen stellt der Alberich Interpreter auch selbstverwaltete Variablen mit Informationen über die Laufzeitumgebung zu Verfügung. Dazu zählen unter anderem das aktuelle Arbeitsverzeichnis, der Pfad, an dem sich das aktuell evaluierte Skript befindet, und der Pfad, an dem sich der Einstiegspunkt des Interpreters befindet, also das Skript was primär ausgeführt wird. Dabei handelt es sich um die Metavariablen `__CWD__`, `__script__` und `__proc__`. Eine weitere Metavariable die intern für Main- und Include-Guards benutzt wird, ist die Variable `__ScriptCalledAs__`. Diese Metavariablen werden dabei je nach evaluiertem Skript angepasst, dass bedeutet, dass sie vor der Evaluierung des Skriptes gesetzt und danach zurückgesetzt werden. Wenn ein eingebundenes Skript also Funktionen registriert, die Werte von Metavariablen abfragen, sind diese unter Umständen bereits zurückgesetzt worden, wenn die registrierte Funktion ausgeführt wird. Aus diesem Grund müssen Metavariablen, deren aktuelle Werte den skriptspezifischen Kontexten auch nach der Evaluierung des Skriptes zur Verfügung stehen sollen, gespeichert werden. Da alle Skripte, die im Rahmen einer Interpretierung evaluiert werden, in dem gleichen globalen Scope arbeiten, muss jedes Skript entweder eigene Speichervariablen implementieren, die anders benannt sind als die der anderen Includes oder die Werte der Metavariablen in eigenen statischen Scopes speichern. Die einzigen Scopes die nach Evaluierung nicht gelöscht werden und damit statisch erzeugbar sind, sind Pseudonamespaces. Algorithmus 40 zeigt wie Metavariablen in einem Pseudonamespace gespeichert werden können.

Algorithmus 40: Abspeichern von Metavariablenwerten in einem Pseudonamespace

```
assert(calledAs(__Include__), __script__ + " is an Include and shouldn't be executed directly")

struct locals{

  // Abspeichern der Werte der Metavariablen bei Evaluierung
  static {
      string [script, CWD] = __script__, __CWD__
  }
}

// Zugriff auf abgespeicherte Werte von Metavariablen
decl logScriptPath(){ slog(locals >> script) }
```

4.1.2.16 Diagnostik

Um die Skripte von Alberich-Distributionen zu debuggen, gibt es mehrere Funktionen, die eine Übersicht über aktuelle Register, Variablen und Evaluierungsergebnisse geben. So kann auch ohne implementierten Language Support effizient mit den Sprachen gearbeitet werden. Die Tabelle 4 führt die wichtigsten Diagnosemethoden auf.

Tabelle 4: Freigeschaltete Diagnostikfunktionen

Funktion	Beschreibung
`log(...)`	Einfache Ausgabe beliebiger Ausdrücke
`slog(...)`	Log mit eingeschobenen Leerzeichen
`logRes(...)`	Ausgabe des erhaltenen Evaluierungsergebnisses
Ausdrucksanalyse	
`isRvalue(...)`	Prüft, ob ein Ausdruck ein R-Value ist
`isLValue(...)`	Prüft, ob ein Ausdruck ein L-Value ist
`isReference(...)`	Prüft, ob ein Ausdruck eine Referenz ist
`isInstance(...)`	Prüft, ob ein Ausdruck eine Instanz ist
`isUniform(...)`	Prüft, ob Ausdruck eine Uniforminstanz / -referenz ist
`variableExists(...)`	Prüft, ob Variable für übergebenen Namen existiert
`typename(...)`	Gibt Typenkeyword für Typen einer Variablen zurück
`typeid(...)`	Gibt Typenindex für Typen einer Variablen zurück
`size(...)`	Gibt die Größe einer Variablen zurück
Registerübersicht	
`logRegisteredBackends()`	Listet alle registrierten Backends
`logTypeRegister()`	Gibt das Typregister aus
`logNullRefs()`	Listet alle registrierten Null-Referenzen
`logFunctionRegister()`	Listet alle registrierten Funktionen
`logStaticFunctionRegister()`	Listet alle registrierten statischen Funktionen
`logMemberFunctionRegister()`	Listet alle registrierten Member-Funktionen
Scopeübersicht	
`logScope()`	Gibt lokalen Scope mit allen Variablen aus
`logRootScope()`	Gibt globalen Root-Scope aus

4.1.2.17 Fehlerbehandlung

Alberich schließt die Fehlerkontextfunktion für interpreterspezifische Fehlermeldungen an. Sorgt die Distribution für den Anschluss der Funktion an die backendspezifische Fehlerbehandlung, wird zu jeder Fehlermeldung ein skriptbezogener Kontext generiert und unter die eigentliche Fehlerausgabe geschrieben. Der Kontext gibt dabei den spezifischen AST-Knoten an, bei dessen Evaluierung der Fehler aufgetreten ist. Die Angabe erfolgt dabei über Skript und Zeile des Knotens und sorgt für eine formatierte Ausgabe der betroffenen Zeilen. In vielen Codeeditoren genügt ein Klick auf die Ausgabe von Skriptpfad und Zeilennummer, um die entsprechende Stelle zu inspizieren. Die Fehlerausgabe des Interpreters liefert dabei zusätzlich die Stelle im Quellcode, an der die Fehlermeldung auftritt, und eine Fehlerbeschreibung. Algorithmus 41 zeigt eine mit einer Alberich-Distribution erzeugte Fehlerausgabe. Die ersten beiden Zeilen werden von der im Interpreter verwendeten Assertion erzeugt. Die letzten beiden Zeilen werden von der angeschlossenen Fehlerkontextfunktion angehängt.

Algorithmus 41: Beispielhafte Interpreterfehlermeldung

```
[2026-03-06-13-10-08] Assertion Failed : 'Mehrfacher Include von ../Import/pckg/functions.bld'
>> Source : src/RegistryLanguage/Interpreter.cpp:280 >> executeScript
>> Context : Import/0_0_0_testEnv.bld:13
13   | fetch script ["pckg/preDecls", "pckg/functions", "pckg/functions"];
       ^~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
```

4.2 Baldur

In diesem Kapitel wird gezeigt, wie die Sprache Baldur genutzt werden kann, um mit algebraischen und indexnotierten Tensorausdrücken zu arbeiten. Da die Sprache Baldur eine Alberich-Distribution ist, folgt sie den in Kapitel 4.1.2 vorgestellten Formatierungs-, Syntax- und Semantikformaten.

4.2.1 Installation

Die Installation von Baldur erfolgt über den Download der im GitHub Repository herausgegebenen Releases. Hierüber lassen sich Interpreter und Languageserver herunterladen. Dabei kann der Interpreter vollständig ohne Einbettung genutzt werden. Der Language Server ermöglicht den Baldur-Sprachsupport in LSP-fähigen Editoren oder IDEs. Allerdings geht damit ein editor- oder IDE-spezifischer Konfigurationsaufwand einher. Für den Codeeditor Visual Studio Code gibt es bereits eine fertige Extension. Diese lässt sich ebenfalls aus den GitHub Releases herunterladen und über `code --install-extension path/to/baldurExtension.vsix` installieren.

4.2.2 Konstruktion algebraischer Tensorausdrücke

Die freigeschalteten algebraischen Tensorausdrücke werden für das Keyword `tExpr` registriert. Das Programm implementiert zudem mehrere Pseudokonstruktoren. Darunter befinden sich Konstruktoren für konkrete algebraische Instanzen und die verschiedenen algebraischen Templatetypen. Bei der Instanzkonstruktion von algebraischen Objekten ist die Übergabe der Tensordimensionen optional. Werden algebraische Ausdrücke ohne Dimensionen konstruiert, werden die Dimensionen bei deren weiterer Behandlung nicht mehr berücksichtigt. Tabelle 5 gibt eine Übersicht über die beschriebenen Pseudo- und Templatekonstruktoren.

Tabelle 5: Konstruktion von externen, algebraischen Tensorausdrücken

Funktion	Beschreibung
`tExpr(label, order)`	Konstruiert algebraischen Tensorausdruck ohne Dimensionsberücksichtigung
`tExpr(label, order, dims)`	Konstruiert algebraischen Tensorausdruck mit Dimensionsberücksichtigung
`tExpr(value)`	Konstruiert algebraischen, skalaren, konstanten Tensorausdruck
`tExprTmpl(label)`	Erstellt Standardtemplate Knoten
`tExprTmpl(label, order)`	Erstellt Standardtemplate Knoten mit konkretisierter Stufe
`tExprArgTmpl(label)`	Erstellt Argtemplate Knoten
`tExprArgTmpl(label, order)`	Erstellt Argtemplate Knoten mit konkretisierter Stufe
`tExprInstTmpl(label)`	Erstellt Instanztemplate Knoten
`tExprCnstTmpl()`	Erstellt Konstantentemplate Knoten
`tExprCnstTmpl(label)`	Erstellt gelabelten Konstantentemplate Knoten

4.2.3 Verknüpfung algebraischer Ausdrücke

Die Verknüpfung der konstruierten externen Knoten erfolgt über algebraische Operationen. Tabelle 6 zeigt eine Übersicht über die implementierten algebraischen Operationen und ihre Nutzung.

Tabelle 6: Implementierte algebraische Operationen

Funktion	Verwendung
Addition	`+=, +`
Subtraktion	`-=, -`
Skalarmultiplikation	`*=, *`
Skalarprodukt	`.=, .`
Doppeltes Skalarprodukt	`..=, ..`
Doppeltes Skalarprodukt	`:=, :`
n-tes Skalarprodukt	`contract(tExpr, tExpr, int, bool)` `contractAssign(tExpr, tExpr, int, bool)`
Kreuzprodukt	`\x=, \x`
Dyadisches Produkt	`\(x)=, \(x)`
Ableitung	`\diff=, \diff` `diff(tExpr, tExpr), diff(tExpr, tExpr, int)`
Negierung	`-`
Transposition	`', transpose(tExpr)`
Inverse	`~, inverse(tExpr)`
Determinante	`det(tExpr)`
Spur	`trace(tExpr), trace(tExpr, int)`
Identitätstensor	`identity(tExpr)`
Nulltensor	`zeros(tExpr)`
Einsentensor	`ones(tExpr)`
Wurzel	`sqrt(tExpr)`
Vorzeichenfunktion	`signum(tExpr)`
Macaulaynorm	`macaulay(tExpr)`
Frobeniusnorm	`frobenius(tExpr)`

Es werden sowohl Funktionen als auch Operatoren angeführt, die algebraische Ausdrücke über die jeweilige Operation verknüpfen. Die Operationen, die zwei Operatoren aufführen überladen diese jeweils binär und verkettend. Dabei wird jeweils der Operator, der das Token = enthält, binär überladen und modifiziert damit den linken Operanden vor Ort. Die Operationen, die nur einen Operator anführen, überladen diesen unär. Grundsätzlich erwarten die Überladungsfunktionen algebraische Ausdrücke als Inputargumente. Nur die Addition, Subtraktion und Skalarmultiplikation können einen algebraischen Tensoroperanden mit einem Integer- oder Double-Operanden verknüpfen. Dabei wird der Integer- oder Double-Operand in einen skalaren, konstanten, algebraischen Tensorausdruck umgewandelt. Ist der linke Operand die Integer- oder Doublevariable, kommt es zu einem dynamischen Recast. Das heißt, dass der linke Operand vor Beaufschlagen der algebraischen Operation vor Ort zu einem Tensorausdruck umgewandelt wird. Dabei erhält der Ausdruckskonstruktor den Wert des ursprünglichen linken, skalaren Operanden übergeben und rekonstruiert diesen damit als skalarkonstanten Tensorausdruck. Die Funktionen, die die gegebenen Operatoren überladen, können auch manuell aufgerufen werden, sind hier

aber nicht konkret aufgeführt. Die Überladung kann dabei jederzeit über die `__LPECONFIG.JSON` nachvollzogen oder angepasst werden. Zudem sind die angegebenen Funktionen mit einer exemplarischen Inputparametersektion angeführt, die die Typenkeywords der erwarteten Inputs enthält. Die Funktionen, die nach dem Schema `xAssign` benannt sind, führen die entsprechende Operation `x` auf dem ersten Operanden durch. Ist die gegebene Funktion nicht nach diesem Schema benannt, gibt sie das Ergebnis der Operation zurück. Gebaute Ausdrücke können dabei jederzeit über die Funktion `rebuild` nachgebaut werden.

4.2.4 Konventionssteuerung

Die Steuerung der Konvention, die beim Bau der Ausdrücke verwendet wird, geschieht über den freigeschalteten, uniformisierten Bool `g_unwrapOperands` oder die Funktionen `setUnwrapOperands` und `getUnwrapOperands`, die diesen Bool intern verwalten. Wird `g_unwrapOperands` dabei auf `true` gesetzt, wendet das Backend beim Bau der Ausdrücke die entpackte Konvention an. Durch das Setzen der Flag auf `false` wird die gepackte Konvention verwendet. Baldur implementiert zusätzlich die Funktionen `wrap` und `unwrap`, die diesen Vorgang für den Neuaufbau eines einzelnen Ausdrucks verpacken und ihn so in die entsprechenden Zielkonventionen überführen. Zudem kann die Konvention eines Ausdrucks über die Funktionen `isWrapped` und `isUnwrapped` überprüft werden. Tabelle 7 gibt eine Übersicht über die genannten Funktionen, die zur Steuerung der verwendeten Konvention und zur direkten Überführung genutzt werden können.

Tabelle 7: Freigeschaltete Funktionen zur Konventionssteuerung

Funktion	Verwendung
`setUnwrapOperands(bool)`	Stellt verwendete Ausdruckskonvention ein
`getUnwrapOperands()`	Gibt eingestellte Konvention zurück
`unwrap(tExpr)`	Überführt Ausdruck in entpackte Konvention
`wrap(tExpr)`	Überführt Ausdruck in gepackte Konvention
`tExpr->isUnwrapped()`	Überprüft ob Ausruck in entpackter Konvention vorliegt
`tExpr->isWrapped()`	Überprüft ob Ausruck in gepackter Konvention vorliegt

4.2.5 Ersetzungen

Baldur implementiert die Attributsfunktionen `replace` und `getReplaced`, die es ermöglichen, Teile eines Ausdrucks zu ersetzen. Die Funktion `replace` führt die Ersetzung dabei in der aktuellen Instanz durch und `getReplaced` gibt ein Kopie des Ausdrucks zurück, in dem die Ersetzungen stattgefunden haben. Die Funktionen können in Tabelle 8 eingesehen werden.

Tabelle 8: Freigeschaltete Funktionen für Ausdruckssubstitution

Funktion	Verwendung
`tExpr->replace(tExpr, tExpr)`	Ersetzung innerhalb der Instanz
`tExpr->getReplaced(tExpr, tExpr)`	Zurückgeben einer Instanz mit Ersetzung

4.2.6 Templatesystem

Das Arbeiten mit den Templatesystem erfordert die Konstruktion und Verknüpfung von externen template-ierten Knoten. Die Überprüfung der Überladung und der Fähigkeit eines Templates einen anderen Ausdruck darzustellen, kann anhand der Überladungsfunktionen für die Operatoren < und == überprüft werden. Algorithmus 42 zeigt die Einsatzmöglichkeiten dieser Diagnostikfunktionen.

Algorithmus 42: Übersicht über Templatediagnostikfunktionen

```
1 int tmplID
2 tExpr [tmplScalarA, tmplScalarB] = tExprTmpl(string(tmplID++), 0), tExprTmpl(string(tmplID++), 0)
3 tExpr [tmplTensorA, tmplTensorB] = tExprTmpl(string(tmplID++)), tExprTmpl(string(tmplID++))
4
5 tExpr [A, B, C] = tExpr("A", 2), tExpr("B", 2), tExpr("C", 2)
6
7 // Abgleich der Fähigkeit des Templates den Ausdruck darzustellen
8 slog(tmplScalarA * tmplTensorB + tmplScalarB * tmplTensorA == 0.5 * A + A * 2.3)
9
10 // Abgleich der Überladung. Einer der beiden Überladungsfunktionsaufrufe muss
11 // 'true' und der andere 'false' zurückgeben, ansonsten kann nicht überladen werden
12 // Wird ein Template konsistent als kleiner eingestuft, wird es im Register
13 // weiter nach vorn sortiert und überlädt damit das andere
14 // In diesem Fall überlädt '(tmplScalarA + tmplScalarB) * tmplTensorA'
15 // das andere Template 'tmplScalarA * tmplTensorA', da es dieses spezifiziert
16 slog(tmplScalarA * tmplTensorA < (tmplScalarA + tmplScalarB) * tmplTensorA)
17 slog((tmplScalarA + tmplScalarB) * tmplTensorA < tmplScalarA * tmplTensorA)
18
19 // Ausgabe : 'true, false, true'
```

4.2.7 Ableitungen

In die Ableitungslogik des Backends lassen sich dynamisch konkrete Ableitungen und Ableitungsregeln implementieren. Die Implementierungen erfolgen über die Funktionen `setDiff` und `setDiffTemplate`. In das Ableitungsregister dürfen nur von konkreten Instanzen abhängige Ausdrücke implementiert werden. Eine Implementierung konkreter Instanzen ist theoretisch auch in das Ableitungsregelregister möglich. Die Registrierungen erfolgen mit den Angaben der Ableitungsoperanden und des Zielausdrucks. Dabei müssen alle template-ierten Knoten, die im Zielausdruck der Ableitungsregel auftauchen, bereits in den Ableitungsoperanden der Regel vorhanden sein. Zudem müssen die Ableitungen, als auch die Ableitungsregeln Zielausdrücke implementieren, deren Stufe und Dimensionen mit den erwarteten Ergebnisgrößen der Ableitungsoperation übereinstimmen. Die Register lassen sich in Echtzeit über die Funktionen `logPresetDiffs` und `logDiffTemplates` ausgeben. Zudem können implementierte Ableitungen oder Ableitungsregeln jederzeit wieder entfernt werden. Tabelle 9 zeigt implementierte Ableitungs-, Registrier- und Ausgabefunktionen. Ableitungen von Ausdrücken sind je nach gewählter Konvention und Konfiguration unterschiedlich aufwendig und zeitintensiv. Für ein nur spärlich implementiertes Ableitungs- und Regelsystem ist die Ableitung von Ausdrücken der gepackten Konvention häufig aufwendiger. Das liegt daran, dass durch das Bauen mit maximal zwei Operanden pro Knoten Operandenketten, die in der entpackten Konfiguration innerhalb eines Knotens gespeichert werden könnten, in der gepackten Konvention durch Teilergebnisknoten verschachtelt werden. Die Ableitung dieser verschachtelten Operandenketten über die Produktregel verursacht dabei einen deutlich höheren Rechenaufwand und deutlich mehr benötigte rekursive Aufrufe der Ableitungsfunktion. Die Verwendung der gepackten Konvention kann für ein vollständigeres Regelwerk die

Ableitung jedoch massiv beschleunigen. Da die gepackte Konvention Operandenketten beliebiger Länge binär in Teilausdrücke zerlegt, können Ausdrücke mikroskopischer auf Templateübereinstimmungen überprüft werden und in einem größeren Anteil der auftretenden Fälle durch die dynamisch implementierten Regeln vereinfacht werden. Deckt das implementierte Regelwerk viele vorkommende Ableitungsfälle ab, wird die Anzahl der notwendigen binären Produktregelanwendungen reduziert und der resultierende Ausdruck ist stark vereinfacht und deutlich weniger komplex.

Tabelle 9: Funktionen zur Verwaltung der Ableitungskonfiguration

Funktion	Verwendung
`diff(tExpr, tExpr)`	Führt die Ableitungsoperation durch
`setDiff(tExpr, tExpr, tExpr)`	Registriert eine konkrete Ableitung
`setDiffTemplate(tExpr, tExpr, tExpr)`	Registriert eine Ableitungsregel
`removeEmplacedDiff(tExpr, tExpr)`	Entfernt registrierte Ableitung
`removeEmplacedDiffTemplate(tExpr, tExpr)`	Entfernt registrierte Ableitungsregel
`logPresetDiffs()`	Ausgabe des Ableitungsregisters
`logDiffTemplates()`	Ausgabe des Ableitungsregelregisters

Die konkrete Formulierung der Ableitungsregeln beeinflusst die benötigte Rechenzeit ebenfalls. Sie entscheidet darüber, wie häufig und tiefgreifend Templatekonsistenzabgleiche durchgeführt werden müssen. Der konkrete Anwendungsfall spielt ebenfalls eine Rolle. Der zeitliche Aufwand, der durch die Formulierung entsteht, muss dabei mit dem geschaffenen Mehrwert in Bezug gesetzt werden. Kann ein allgemein formuliertes Template häufig angewendet werden, wird die Evaluierungszeit zwar gestreckt, der resultierende Ausdruck ist durch die häufige Anwendung jedoch stark vereinfacht und kann deshalb beschleunigt weiterverarbeitet werden. Sind in Folge einer allgemeinen Formulierung jedoch häufige, aufwendige Konsistenzabgleiche nötig, die in einem Großteil der Fälle negativ ausfallen, treibt die Formulierung die Evaluierungszeit massiv nach oben, ohne eine verhältnismäßige Vereinfachung zu erzielen. Das bedeutet, dass die Formulierung einen nicht nennenswerten Vorteil für die Weiterverarbeitung des Ausdrucks schafft, die Evaluierungszeit jedoch massiv verlangsamt. In einem solchen Fall sollte eine weniger allgemeine und aufwendige Formulierung gewählt werden, da die allgemeinere keinen wirklichen Mehrwert schafft. Algorithmus 43 zeigt die exemplarische Nutzung der registergestützten Ableitungskonfiguration.

Algorithmus 43: Ableitungs-Beispiel

```
1 // Die Label sind hier irrelevant, sie dienen nur der Unterscheidbarkeit der Templateknoten
2 int tmplID
3 tExpr tmplScalar = tExprTmpl(string(tmplID++), 0)
4 tExpr [tmplTensor, tmplOtherTensor] = tExprTmpl(string(tmplID++)), tExprTmpl(string(tmplID++))
5
6 tExpr [a, A, B, C] = tExpr("a", 0), tExpr("A", 2), tExpr("B", 2), tExpr("C", 4)
7
8 // Übergabe der Vorschrift als : (Ableitungsoperand 1, Ableitungsoperand 2, Zielausdruck)
9 setDiff(A, B, C)
10
11 // Deckt aufgrund von Permutationsabgleichen für assoziative Operationen sowohl
12 // 'tmplScalar * tmplTensor' als auch 'tmplTensor * tmplScalar' ab
13 setDiffTemplate(tmplScalar * tmplTensor, tmplOtherTensor, \
14                tmplScalar * diff(tmplTensor, tmplOtherTensor))
15
16 slog(diff(A * a, B))
17 // Ausgabe : 'a * C'
```

4.2.8 Vereinfachungen

Baldur implementiert eine dynamisch konfigurierbare Vereinfachungslogik. Die Vereinfachung lässt sich über die Funktionen `simplify` und `simplified` durchführen. Die Angabe der Vereinfachungsregeln erfolgt über die Funktion `setEqual`. In das Simplifizierungsregister wird ein Ausdruckspaar bestehend aus einem Basisausdruck und einem Zielausdruck implementiert. Dabei können die Ausdrücke sowohl von Templates, als auch ausschließlich von Instanzen abhängig sein. Registrierte Simplifizierungen können ebenfalls gelöscht werden. Zudem kann das Simplifizierungsregister jederzeit nach Überladung ausgegeben werden. Tabelle 10 stellt eine Übersicht über implementierte Simplifizierungs-, Registrier- und Ausgabefunktionen bereit.

Tabelle 10: Funktionen zur Steuerung der Vereinfachungskonfiguration

Funktion	Verwendung
`simplify(tExpr)`	Vereinfacht Audruck vor Ort
`simplified(tExpr)`	Gibt vereinfachten Ausdruck zurück
`setEqual(tExpr, tExpr)`	Registriert eine Simplifizierung
`removeEqual(tExpr)`	Löscht eine registrierte Simplifizierung
`logSimplifications()`	Gibt Vereinfachungsregister nach Überladung aus

Die Vereinfachungen sollten mit gepackten Ausdrücken durchgeführt werden. Zum einen ist das Aufstellen der Regeln für beliebig lange Operandenketten praktisch nicht möglich und effizient, zum anderen ermöglichen gepackte Ausdrücke eine tiefgreifendere strukturelle Analyse und einen mikroskopischeren Abgleich mit den hinterlegten Templates. Beim Simplifizieren muss zudem darauf geachtet werden, dass die entsprechende Konvention gesetzt ist, in der der vereinfachte Ausdruck ausgegeben werden soll. Die Konvention des Resultatausdrucks entspricht nicht der Eingabe, sondern der eingestellten Konvention. Das liegt daran, dass die Simplifizierung die vereinfachten Ausdrücke nach jedem Schritt neu baut und auf die eingestellte Konvention zurückgreift.
Die Vereinfachung ist der performancekritischste Teil des Backends. Ein ausführliches Vereinfachungsregelwerk bietet die Möglichkeit, bestehende Ausdrücke bei gleichbleibender Aussage

deutlich zu komprimieren. Die weitere Arbeit mit den komprimierten Ausdrücken ist durch ihre verringerte Größe weniger aufwendig und zeitintensiv. Im Falle eines Exports wird der Aufwand für die Umgebung, in die exportiert wird, ebenfalls verringert. Der Umfang und die Formulierung des Vereinfachungsregelwerks bestimmt die Effizienz der durchgeführten Vereinfachungen. Sehr allgemein formulierte Templates benötigen beim Abgleich mit großen Ausdrücken tiefgreifende, aufwendige Konsistenzabgleiche. Templates, die zu spezifisch formuliert werden, decken unter Umständen nicht genügend Anwendungsfälle ab. Die Formulierung eines Vereinfachungstemplates sollte deshalb immer genau abgewogen und der Einfluss der Formulierung auf die Laufzeit und Ergebnisse des Programms überprüft werden. Die Implementierung der Regel `setEqual(StdTmpl + StdTmpl, 2 * StdTmpl)` beispielsweise, kann die Programmausführung für die Vereinfachung großer, gepackter Ausdrücke massiv verlangsamen. Das liegt daran, dass jede Addition des gepackten Ausdrucks für diese Regel zunächst auf Konsistenz geprüft werden muss. Dabei wird der erste Operand rekursiv so lange mit dem zweiten Operanden verglichen, bis ein Unterschied festgestellt werden kann. Enthalten beide Operanden assoziative Operationen, müssen unter Umständen zusätzlich Abgleiche aller möglichen Permutationen durchgeführt werden. Der Ausdruck `setEqual(ArgTmpl + ArgTmpl, 2 * ArgTmpl)` deckt zwar deutlich weniger potenzielle Anwendungsfälle ab, kann aber in einem Bruchteil der Zeit abgeglichen und auf Konsistenz überprüft werden. Die Auswahl der Formulierung für diese Vereinfachungsregel muss fallspezifisch getroffen werden. Im Rahmen der getesteten Fälle verlangsamt das allgemein formulierte Template die Programmausführung deutlich und trägt effektiv nur minimal zur Vereinfachung bei. Algorithmus 44 zeigt die exemplarische Nutzung der registergestützten Vereinfachungskonfiguration.

Algorithmus 44: Vereinfachungsbeispiel

```
1  int tmplID
2  tExpr [tmplScalar, tmplTensor] = tExprTmpl(string(tmplID++), 0), tExprTmpl(string(tmplID++))
3  tExpr [zerosInstanceTemplate]  = tExprInstTmpl("zeros")
4
5  tExpr [A, B, C] = tExpr("A", 2), tExpr("B", 2), tExpr("C", 2)
6  tExpr [a, b, c] = tExpr("a", 0), tExpr("b", 0), tExpr("c", 0)
7
8  setEqual(0 + tmplScalar, tmplScalar)
9  setEqual(zerosInstanceTemplate + tmplTensor, tmplTensor)
10 setEqual(0 * tmplTensor, zeros(tmplTensor))
11 setEqual(1 * tmplTensor, tmplTensor)
12
13 tExpr expr = a * A + b * B + c * C
14 expr->replace(a, tExpr(0))
15 expr->replace(b, tExpr(1))
16 expr->replace(c, tExpr(1))
17
18 slog(simplified(expr))
19 // Ausgabe : 'B + C'
```

4.2.9 Indexnotierte Tensorausdrücke

Die indexnotierten Ausdrücke werden für das Typekeyword `tIdn` registriert. Die Umwandlung eines algebraischen Ausdrucks zu einem indexnotierten erfolgt über die Funktion `toIDN`. Dabei ist es wichtig sicherzustellen, dass die aktuell eingestellte Konvention mit der des Ausdrucks übereinstimmt, der konvertiert wird. Algorithmus 45 demonstriert die Benutzung der Konvertierung.

Algorithmus 45: Konvertierung von algebraischen zu indexnotierten Ausdrücken

```
1 tExpr [A, B] = tExpr("A", 2), tExpr("B", 2)
2 tExpr [a, b] = tExpr("a", 0), tExpr("b", 0)
3
4 tExpr expr = a * A + b * B
5
6 // Assertion hier redundant
7 assert(expr->isUnwrapped() == getUnwrapOperands(), "...")
8 tIdn idnExpr = toIDN(expr)
9
10 slog(idnExpr)
11 // Ausgabe : 'a[] * A[2,3] + b[] * B[2,3]'
```

4.2.10 Julia-Skript-Generierung

Die Julia-Skript-Generierung geschieht über die Attributsfunktion `toJuliaString` des indexnotierten Ausdrucks. Diese erzeugt einen String, der dann in eine Datei geschrieben werden kann. Bei der Julia-Skript-Generierung empfiehlt sich die Verwendung eines gepackten, indexnotierten Ausdrucks. Für große entpackte Ausdrücke kann die Anzahl der Abhängigkeiten eines Knotens bereits die maximale zulässige Komplexität übersteigen, bei der die Substitution der Abhängigkeiten forciert wird. Da die Anzahl der Abhängigkeiten diesen Maximalwert bereits übersteigt, kann auch die Substitution aller Abhängigkeiten nicht dazu führen, dass die Komplexität des Ausdrucks die maximal zulässige unterschreitet. Ein solcher Fall führt zur Terminierung der Julia-Skript-Generierung. Zusätzlich werden die für gepackte Ausdrücke generierten Skripte schneller evaluiert. Algorithmus 46 demonstriert die Generierung eines Julia-Skripts für einen algebraischen Ausdruck.

Algorithmus 46: Beispiel für Julia-Skript-Generierung

```
1 int nComps = 3
2 tExpr [A, B] = tExpr("A", 2, args(nComps, nComps)), tExpr("B", 2, args(nComps, nComps))
3 tExpr [a, b] = tExpr("a", 0), tExpr("b", 0)
4
5 tExpr expr = a * A + b * B
6
7 setUnwrapOperands(false)
8 expr = wrap(expr)
9
10 assert(expr->isUnwrapped() == getUnwrapOperands(), "...")
11 tIdn idnExpr = toIDN(expr)
12
13 string jlScript = idnExpr->toJuliaString("evalExpr")
14 jlScript->writeToFile("../Import/juliaScripts/expr.jl")
15
16 slog(jlScript)
17 // Ausgabe : '
18 // ...
19 // function evalExpr(a, A, b, B)
20 //  ...
21 //      @tensor opt=true res[idx2, idx3] := ((a * A[idx2, idx3]) + (b * B[idx2, idx3]))
22 //      return res
23 // end
24 // '
```

4.2.11 Potenzielle Limitierungen

Die Faktoren, die das Arbeiten mit Baldur und dem freigeschalteten Tensoralgebrabackend limitieren, sind der verfügbare Arbeitsspeicher und der verfügbare Stack des Programms. Werden die algebraischen Ausdrücke größer als der aktuell verfügbare Arbeitsspeicher, kommt es zum Absturz des Programms. Der zur Verfügung stehende Stack ist ebenfalls ein potenziell limitierender Faktor. Bei jedem rekursiven Funktionsaufruf wird ein neuer Stackframe angelegt, der unter anderem Informationen wie die Rückgabeadresse und lokale Variablen enthält. Dieser belegte Speicherplatz kann erst nach Rückgabe des rekursiven Funktionsaufrufs freigegeben werden. Damit limitiert die Stackgröße nicht die Größe des Ausdrucks, sondern die mögliche Rekursionstiefe. Da für große Ausdrücke bereits nicht performancekritische Standardoperationen sehr hohe Rekursionstiefen erreichen können, kommt es teilweise zum Überlauf des Stacks. Bei einem Überlauf wird das Programm ohne Fehlermeldung terminiert. Eine Möglichkeit den verfügbaren Stack zu vergrößern ist das Kompilieren des Interpreters mit der konkreten Angabe der gewünschten Stackgröße. Diese Variante ist allerdings nicht laufzeitfähig. Alberich implementiert mit den Standardoperationen eine alternative Option über die Methode `callOnExtendedStack`. Dabei wird ein Funktionsaufruf in einem temporären, nativen Windowsthread abgesetzt, der mit einer explizit angegeben Stackgröße konstruiert und nach der Funktionsausführung gelöscht wird.

4.2.12 Fehlerbehandlung

Baldur implementiert für das Tensoralgebrabackend eine umfassende Fehlerausgabe, die dem Nutzer konkret schildert, warum es zu Fehlern kommt. In einigen Fällen kann es aber auch zum Absturz oder Einfrieren des Interpreters kommen. Abstürze lassen sich meistens anhand einer Kollision mit limitierenden Faktoren erklären. Das Einfrieren des Programms kann im Rahmen der Simplifizierung auftreten. Dazu kommt es, wenn Regeln untereinander eine Art Dauerumformungsschleife erzeugen. Diese ist dabei nicht zwangsläufig offensichtlich. Solche Schleifen können sich häufig hinter vielen sehr allgemeinen Regelformulierungen verbergen. Um diese kollidierenden Formulierungen zu finden, müssen die Regelwerke Stück für Stück untersucht und in verschiedenen Versionen für den Fall getestet werden, in dem es zum Einfrieren kommt.

5 Leistungsbeurteilung

In diesem Kapitel wird die Leistungsfähigkeit Baldurs bewertet. Zur Beurteilung wird eine Reihe von konkreten Anwendungen implementiert und die Evaluierungszeiten der Baldur Skripte ausgewertet. Implementiert werden mehrere Skripte, die die Tangenten für die Spannung und nicht linearen Dehnungsanteile verschiedener Materialmodelle für die TSM (time-separated stochastic mechanics) evaluieren und als Julia-Skript exportieren. Dazu wird eine minimalistische Umgebung aufgesetzt, die eine Basiseinrichtung der Templateregister vornimmt und Grundfunktionen zur Verfügung stellt. Die Umgebung und die beurteilten Implementierungen werden über das Baldur GitHub Repository zur Verfügung gestellt.

5.1 Auswertung Beispielimplementierungen

Die Evaluierung der Beispielimplementierungen zeigt, dass Baldur auch komplexere Ausdrücke auf handelsüblicher Hardware in kurzer Zeit verarbeiten kann. Konkrete Evaluierungszeiten hängen dabei ausschlaggebend von der im Skript vorgenommenen Optimierung ab. Zu der Optimierung zählt dabei die anwendungsspezifische Wahl der Ausdruckskonvention und der Regelwerke. Die implementierte Umgebung bietet eine Grundlage, die entweder als Plug-and-Play-Lösung genutzt werden kann oder als Grundstock, der durch eigene fallspezifische Implementierungen oder Modifizierungen erweitert wird. Die alleinige Nutzung dieser Umgebung erzielt innerhalb der meisten implementierten Anwendungen bereits sehr solide Evaluierungszeiten und Ergebnisse. Diese können jedoch durch fallspezifische Anweisungen noch deutlich optimiert werden. Tabelle 11 zeigt die Statistiken einer durchgeführten Evaluierung der viskoplastischen Tangenten für die TSM 0. bis 6. Ordnung auf einem Computer mit Intel Core i7 und 32 GB RAM.

Tabelle 11: Evaluierungsstatistiken nach TSM-Stufe

Stufe []	Evaluierungszeit [s]	Größter Ausdruck vor Vereinfachung [Nodes]	Größte exportierte Tangente [Nodes]	Längstes Julia Tangentenskript [Zeilen]
0	0,02	60	50	40
1	0,02	800	250	130
2	0,25	5 500	1 400	500
3	3	65 000	8 600	2 400
4	40	680 000	60 000	13 000
5	660	8 000 000	450 000	85 000
6	12 300	110 000 000	3 600 000	635 000

5.2 Vergleich mit vorhandenen Implementierungen

Zur weiteren Leistungsbeurteilung werden eine Baldur- und eine AceGen-Implementierung miteinander verglichen. Beide Implementierungen ermitteln die TSM-Tangenten des viskoplastischen Materialmodells für die TSM zweiter Ordnung und exportieren diese als Julia-Skripte. Die Baldur-Implementierung befindet sich im Baldur-GitHub-Repository. Die AceGen Implementierung wurde zur Erzeugung der in einer institutsinternen Studienarbeit genutzten TSM-Tangenten-Skripten verwendet. Der Vergleich umfasst dabei mehrere beim Evaluieren der Implementierungen ermittelten Leistungsmerkmale. Zudem werden die erzeugten Tangentenskripte im Rahmen einer der genannten Studienarbeit implementierten Benchmark-Zeitschrittsimulation

des viskoplastischen Materialmodells auf Materialpunktebene mit 80 Zeitschritten verglichen. Tabelle 12 gibt eine Übersicht über diesen Vergleich. Dabei fällt auf, dass die Evaluierung der Tangenten mit der Baldur-Implementierung deutlich weniger Rechenzeit in Anspruch nimmt und einen weniger komplexen Ergebnisausdruck erzeugt. Deshalb kann die Benchmark-Simulation mit den von der Baldur-Implementierung erzeugten Tangenten-Skripten in deutlich kürzerer Zeit berechnet werden. Da AceGen ein sehr umfangreiches digitales Ökosystem darstellt und eine umfangreiche, performante Funktionspalette bereitstellt, findet es in der Kontinuumsmechanik breite Anwendung. Dieser spezielle Anwendungsfall kann durch die Verwendung des spezialisierten Baldur-Interpreters jedoch schneller und effizienter gehandhabt werden. Gründe dafür sind die hier vorteilhafte rein symbolisch-algebraische Ausdrucksbehandlung, spezialisierte, minimale, fallspezifisch optimierte Regelwerke und minimale Interpreterabhängigkeiten.

Tabelle 12: Vergleich zwischen AceGen- und Baldur-Implementierungen

Kenngröße	**AceGen-Impl.**	**Baldur-Impl.**
Evaluierungszeit größte Tangente [s]	$> 1\,000$	0,25
Komplexität größte Tangente vor Vereinfachung [Nodes]	$> 670\,000$	5 500
Evaluierungszeit Benchmark-Rechnung [s]	> 200	13

5.3 Zusammenfassung

Baldur ermöglicht das Arbeiten mit symbolischen, algebraischen Tensorausdrücken, deren Umwandlung in Indexnotation und den Export als Julia-Skript. Zudem ist eine sehr flexible Programmsteuerung und fallspezifische Optimierung aus den Baldurskripten heraus möglich. Damit ist der Interpreter in der Lage, komplexere, algebraische Ausdrücke auf handelsüblicher Hardware in annehmbaren Zeiten zu verarbeiten. Die Syntax zum Aufstellen der Ausdrücke orientiert sich dabei stark an der geläufigen Tensoralgebra. Damit wird ein geringer Einarbeitungsaufwand ermöglicht. Zudem kann Baldur über den Download und die Installation einer öffentlich verfügbaren Visual Studio Code Extension unkompliziert installiert werden. Damit erfüllt Baldur die am Anfang dieser Arbeit formulierten Anforderungen m.E. voll und ganz.

5.4 Ausblick

Zur weiteren Verbesserung der Effizienz und Rechenzeit des Programms könnten die rekursiven Evaluierungen sowohl in der Skript-, als auch Ausdrucksverarbeitung durch das Entpacken in Verarbeitungsstacks linearisiert werden. Zudem könnten die stringbasierten Zugriffe auf Scope-Variablen und externe Knoten durch die Verwendung dedizierter Indizierungszuordnungen beschleunigt werden. Die Implementierung fallspezifischer Syntaxkonzepte könnte den Schreibkomfort, sowohl innerhalb der Distributionsherstellung, als auch bei dem direkten Arbeiten mit der Sprache, verbessern. Möglichkeiten, die Funktionspalette Baldurs zu erweitern, wären das Implementieren weiterer Exportformate, wie LaTeX-Formeln oder Python-Skripten, oder der Einbettung einer optionalen, direkten Evaluierung durch das Backend. Zudem könnte Baldur direkt in Laufzeitumgebungen anderer Tools portiert werden, um direkter mit diesen zu interagieren.